Oblig 2b – Nevrale nettverk

Våren 2026

Det er en god idé å lese gjennom hele oppgavesettet før dere setter i gang. Dersom dere har spørsmål så kan dere:

- gå i gruppetime,
- spørre på Discourse
- eller sende epost til in1160-support@ifi.uio.no dersom alternativene over av en eller annen grunn ikke passer for spørsmålet.

Innlevering

Oppgaven leveres innen 25. mars klokken 23.59 i Devilry.

Innleveringen skal bestå av én Jupyter notebook med både kode og tilhørende forklaringer. Før innlevering skal du kjøre gjennom hele notebooken, før du lagrer siste gang. Den bør kjøre uten å feile og vise alt som skal være med.

Vi understreker at innlevering av kode alene ikke er nok for å bestå oppgaven – vi forventer at notebooken også skal inneholde kommentarer (på norsk eller engelsk) på hva dere har gjort og begrunnelser for valgene dere har tatt underveis. La enhver oblig bli en trening i å formidle forskning. Bruk helst hele setninger, og matematiske formler om nødvendig. Resultater skal presenteres i tabeller på en oversiktlig måte. Det å forklare med egne ord, bruke begreper vi har gått gjennom på forelesningene og å forklare og reflektere over løsningene deres er en viktig del av læringsprosessen – ta det på alvor!

Når det gjelder bruk av generative prateroboter (ChatGPT og lignende): Dere kan bruke dem som en "sparringspartner", for eksempel for å forklare noe dere ikke helt har forstått. Dere har imidlertid ikke lov til å bruke dem til å generere løsninger (enten delvis eller fullstendig) til noen av oppgavene. Funksjoner for automatisk skriving av kode, som Copilot i VS Code, må derfor også være deaktivert mens dere jobber på obligen.

Bruker dere KI-verktøy vil vi også at dere kort beskriver hvordan dere har brukt dem under arbeidet med oppgaven.

Det er ikke mulighet for omlevering av obliger som ikke bestås.

Poeng: Obligen gir maksimalt 25 poeng. For å bestå kreves det totalt 30 av 50 poeng fra oblig 2a og 2b tilsammen. Poengfordelingen er markert i overskriften til oppgaven med `p`.

Merk: Obligene er nye, så vi oppdaterer skrivefeil og liknende fortløpende. For å få den nyeste endringene kan dere kjøre `git pull` i terminalen (gitt at dere har klonet repositoriet). Dersom dere har endret en fil med samme navn kan dere få en *merge conflict*. For å unngå dette kan dere endre navnet på filen og kjøre `git pull` på nytt. En ny fil med det originale navnet vil da dukke opp, der endringene er gjort. For eksempel kan dere endre navn på denne fila fra `oblig_2b.ipynb` til `oblig_2b_<brukernavn>.ipynb`.

Finner dere noe dere tror er feil i obligen, må dere gjerne si ifra på Discourse.

Bakgrunn

I denne obligen skal vi bruke nevrale nettverk til å klassifisere håndskrevne sifre fra det legendariske MNIST datasettet. Dette har vært et av de viktigste datasettene i utviklingen av dyplæring fra starten av 90-tallet til starten av 2010-tallet. Vi skal bruke et enkelt forovermatet nevralt nettverk (feed forward neural network), der blant annet foroverfasen, aktiveringsfunksjoner og evaluering skal implementeres. Obligen ser på hvordan *tilbakepropagering* (backpropagation) og *stokastisk gradientnedstigning* (stochastic gradient descent (SGD)) fungerer, men implementasjonen av dette er ikke en del av obligen og koden for det blir gitt ferdig implementert.

Det er *sterkt anbefalt* å se igjennom de tre første videoene i denne introduksjonen til nevrale nettverk før dere begynner på denne obligen. Denne videoserien av 3Blue1Brown (Grant Sanderson) bruker eksepsjonelle animasjoner til å introdusere bildegjenkjenning, nevrale nettverk og SGD på en pedagogisk og vakker måte. Det vil bli betydelig enklere å fullføre denne obligen med denne kunnskapen. Denne videoen kan også være til nytte.

```
import numpy as np

from plotting import plot_mislabeled_mnist_images,
plot_random_mnist_images, plot_worst_predicted_mnist_images
from tests2b import (
    test_calculate_accuracy,
    test_calculate_multiclass_cross_entropy_loss,
    test_count_parameters,
    test_forward_pass,
    test_predict,
    test_relu_class,
    test_sigmoid_class,
)
from utils2b import IdentityActivation, NeuralNetwork,
integer_one_hot_encode, load_mnist_data, softmax
```

1 - MNIST [Totalt 2p]

Litt historisk kontekst

La oss se litt nærmere på datasettet vi skal bruke. MNIST (Modified National Institute of Standards and Technology database) inneholder 60 000 treningsbilder av håndtegnete sifre annotert med 0-9 (de sanne verdiene), og 10 000 tilsvarende testbilder. Bildene er i svart-hvitt og har en oppløsning på 28 x 28 piksler. Dataene har opprinnelse fra den amerikanske etaten NIST (National Institute of Standards and Technology) og ble prosessert av forskere i AT&T Bell labs på 90-tallet for å gjøre dataene mer standardiserte og konsekvente. Datasettet ble utviklet for å gjøre framgang i gjenkjenning av håndskrevne tall og ord, som var et viktig og stort sett uløst problem på den tiden. Bildene i treningsdataene er skrevet av ansatte i amerikanske byråer, mens bildene i testsetet er skrevet av elever på videregående nivå (high school).

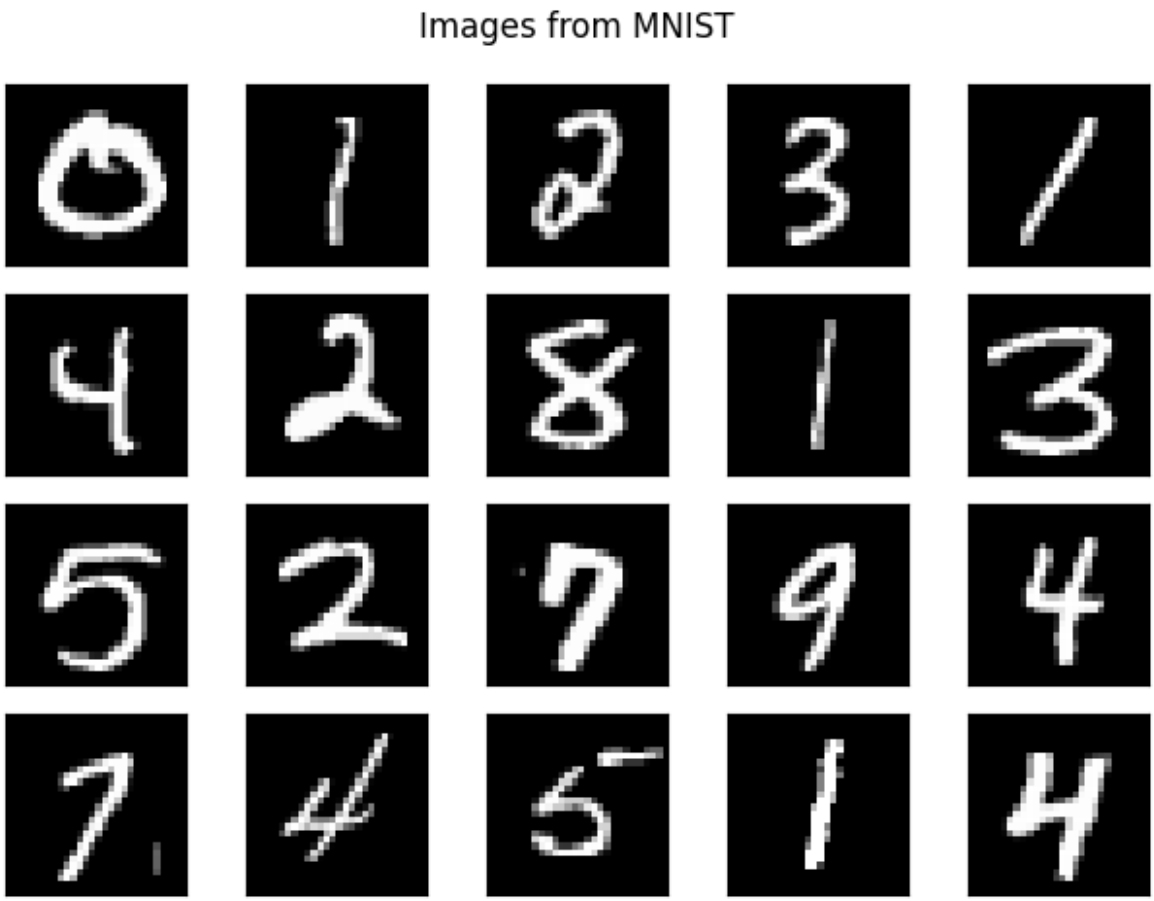
Datasettet førte til en av de tidligste applikasjonene av nevrale nettverk. På denne tiden hadde nevrale nettverk fortsatt et begrenset bruksområde og de dominerende maskinlæringsmetodene baserte seg på manuelt konstruerte trekk (features) og modeller som k-NN, lineære klassifikatorer og support vector machines (SVM). Nevrale nettverk krever ofte mye data for å fungere, ofte mer enn andre maskinlæringsmetoder, og MNIST var en viktig brikke i utviklingen av nevrale nettverk i tiårene etterpå. Allikevel oppnådde ikke dyplæring bred anerkjennelse før i 2012, da det nevrale nettverket *AlexNet* vant bildegjenkjennelseskonkurransen *ImageNet*, som inneholder mye mer realistiske bilder med høyere oppløsning og flere klasser.

MNIST er i dag regnet som et svært enkelt datasett, tilsvarende et "Hello world"-datasett for maskinlæring. Derfor er det ikke lenger brukt for utvikling av dyplæring, siden modeller kan oppnå svært bra resultat selv om de er ineffektive. Dette gjør det heldigvis perfekt for å komme i gang med dyplæring. Vi skal bruke et enkelt forovermatet nevralt nettverk vi lager fra scratch med NumPy og trenger ingen flere fancy metoder for å få det til å fungere. Samtidig er problemet vanskelig nok til at det ikke er trivielt å løse med tradisjonelle og regelbaserte metoder eller trekk konstruksjon (feature engineering).

MNIST-datasettet

La oss se på hvordan dataene i datasettet faktisk ser ut. Funksjonen `load_mnist_data()` fra `utils2b.py` kan brukes til å laste inn datasettet, og det er diverse funksjoner for å plotte det i `plotting.py`. La oss prøve:

```
mnist_data = load_mnist_data(scale_x_data=False)
x_train = mnist_data["x_train"]
y_train = mnist_data["y_train"]

plot_random_mnist_images(images=x_train, n_random=20, n_cols=5,
title="Images from MNIST")
print(f"Shape of dataset: {x_train.shape}")
```




```
Shape of dataset: (50000, 784)
```

Over kan dere se noen eksempler på bildene. Funksjonen `load_mnist_data()` gir både trenings-, validerings- og testdata. Som man kan se fra dimensjonene til `x_train` er det 50 000 bilder i treningsdataene, mens det er 10 000 bilder i validerings- og testdataene. Hvert bilde er representert som en liste av 784 tall, som tilsvarer oppløsningen av 28 x 28 piksler. Hver piksel er et heltall mellom 0 og 255, der 0 representerer helt svart og 255 representerer helt hvitt. Bilder med farge har som regel tre kanaler (channels), et for rødt, grønt og blått, alle med fargestyrke mellom 0 og 255. Om disse tre har lik styrke får man gråtoner, så for svart-hvitt bilder trenger man bare én kanal.

De sanne verdiene inneholder tall mellom 0 og 9 som representerer hvilket siffer som er på det tilsvarende bildet.

Når vi gir dataene som input til nevrale nettverk er det ofte fint å skalere den, slik at man ikke ender opp med for store verdier tidlig i treningen som skaper instabilitet. Det kan man enkelt gjøre ved å dele på 255, slik at alle pikselverdiene blir mellom 0 og 1. Dersom argumentet `scale_x_data=True` blir sendt til `load_mnist_data()` blir dataene skalert slik.

Oppgave 1 [2p]: La oss gjøre noen oppgaver relatert til datasettet for å bli bedre kjent med det.

- a) Hvor mange bilder i treningsettet representerer sifferet "9"?

- b) Hva er den gjennomsnittlige pikselverdien av alle pikslene av alle bildene i treningsdataene?

```
# a) Hvor mange bilder i treningssettet representerer sifferet "9"?

# Sjekker dimensjonene til treningsdataene
print("x_train.shape:", x_train.shape)
print("y_train.shape:", y_train.shape)

# Teller hvor mange bilder som er lik 9
is_nine = (y_train == 9)    # Dette er et boolsk array, der True er
bilder av tallet 9
print("Første 40 verdier i is_nine:", is_nine[:40])

# I numpy behandles True som 1 og False som 0 i summering.
# Ergo, summen gir direkte antall 9-ere.
num_nines = np.sum(is_nine)
print("Antall 9-ere i treningssettet:", num_nines)

# Finner gjennomsnittlig pikselverdi i hele treningssettet, inkludert
alle 50000*784 pikselverdier
mean_pixel_value = np.mean(x_train)
print("Gjennomsnittlig pikselverdi:", mean_pixel_value)
```

```
x_train.shape: (50000, 784)
y_train.shape: (50000,)
Første 40 verdier i is_nine: [False False False False False False
False False False False False False
 False False False False False False False False False False False
False
 False False False False False False False False False False False
True
  True False False False]
Antall 9-ere i treningssettet: 4956
Gjennomsnittlig pikselverdi: 33.339687653061226
```

Forklaring Oppgave 1a)

For å svare på deloppgave a, så undersøkte jeg etikketene i y_train, som inneholder den sanne klassen for hvert bilde i treningssettet. Ved å sammenligne alle etikettene med 9, altså y_train==9, så får vi en boolsk array der hver verdi er True dersom bildet representerer sifferet 9 og False ellers. Når denne boolske arrayen summeres med np.sum(), så blir True tolket som 1 og False tolket som 0. På grunn av denne løsningen, så får vi at summen gir oss antall bilder i treningssettet som representerer sifferet 9.

Forklaring Oppgave 1b)

For å svare på deloppgave b, så ønsker vi altså å finne den gjennomsnittlige pikselverdien i hele treningsdatasettet. Hvert bilde i x_train består av 784 pikselverdier og det er 50 000 bilder i treningssettet. Ved å bruke metoden np.mean(x_train), så beregenes gjennomsnittet av pikselverdiene i hele matrisen. Dette gir oss et mål på hvor lys eller mørk datasamlingen er i

gjennomsnitt. Siden bakgrunnen i MNIST-bildene ofte er mørke og selve sifferet dekker et mindre totalt areal av den totale overflaten, så vil gjennomsnittet naturlig nok være lavt sammenlignet med maksimumsverdien på 255, som i dette tilfellet er på 33.34.

[Bonus]: De første nevrale nettverkene trent på MNIST, kalt *LeNet* (oppkalt etter *Yann LeCun*, som er en av de viktigste forskerne innenfor dyplæring og var med å utvikle MNIST), var *convolutional neural networks* (CNN). Dette er nevrale nettverk som bruker en operasjon kalt *konvolusjoner* i de tidligste lagene i nettverket. Konvolusjoner har lenge blitt brukt i bildebehandling og kan blant annet bli brukt til å påføre filtere, gjøre bilder uskarpe og finne kanter i bilder (alt uten maskinlæring). I CNN-er blir parameterne til konvolusjonen funnet gjennom trening, som gir mange fordeler. I "fully connected feed-forward" nevrale nettverk må vi ha en vekt (parameter) mellom hver av pikselverdiene og hver av nodene i det første skjulte laget. Dette fungerer for MNIST som har svært få piksler, men for mer realistiske datasett blir dette fort upraktisk mange parametere. CNN-er kan bruke langt færre parametere og har andre fordeler som at de bedre utnytter den to-dimensjonale strukturen i bilder. Om dere er interessert i å lære mer kan dere se denne videoen for en visuell forklaring av hvordan konvolusjon fungerer på bilder. Dere kan lære om CNN-er i faget IN3310.

2 - Nevrale nettverk [Totalt 13p]

Vi skal bruke forovermatede nevrale nettverk til å klassifisere bildene i MNIST. Vi har prøvd å lage denne obligen slik at dere vil få en best mulig introduksjon til nevrale nettverk på et detaljert implementasjonsnivå, samtidig som dere skal slippe å knote med for mye med likninger og indekseringer. Derfor har vi valgt å gi dere ferdig kode for de mest intrikate stegene, og har latt de delene vi tror gir best læring for minst mulig knoting være opp til dere å kode. Dere trenger derfor ikke å implementere tilbakepropagering, SGD og vektinitialisering, men dere skal blant annet implementere foroverfasen, prediksjon, aktiveringsfunksjoner og noen funksjoner for evaluering.

2.1 - Aktiveringsfunksjoner [2p]

Vi starter med å implementere aktiveringsfunksjonene vi skal bruke, Sigmoid og ReLU (rectified linear unit). Aktiveringsfunksjoner blir brukt i foroverfasen, mens den deriverte til funksjonene blir brukt i bakoverfasen. Dette er praktisk å implementere som en klasse uten klassevariabler.

La oss se litt på aktiveringsfunksjonene og deres deriverte. Som tidligere nevnt er det ikke pensum å regne ut deriverte uttrykk, men det er pensum å forstå hva de deriverte er og hvorfor vi bruker dem. Kort sagt beskriver den deriverte til en funksjon hvor raskt funksjonen vokser eller minsker i et bestemt punkt. Vi noterer den deriverte av funksjonen $f(x)$ som $f'(x)$. Hvis funksjonen vår er $f(x)=3x+0.5$, er den deriverte funksjonen $f'(x)=3$, fordi $f(x)$ alltid stiger med 3.

I bakoverfasen vil vi derivere tapfunksjonen med hensyn på parameterne, slik at vi kan finne ut hvordan parameterne burde bli justert for at tapet skal minske mest mulig. Siden aktiveringsfunksjonen er med på å predikere (foroverfasen), vil deres deriverte dukke opp i oppdateringsregelen. Dere skal altså ikke implementere oppdateringsregelen (tilbakepropagering og SGD), men implementasjonen dere skal bruke avhenger av at dere implementerer prediksjon og den deriverte til aktiveringsfunksjonene.

Sigmoid, som vi implementerte i oblig 2a i forbindelse med logistisk regresjon, er definert ved følgende:

$$\sigma(x)=\frac{1}{1+e^{-x}}$$

Ved å bruke derivasjonsregelen til ganging eller deling og litt utregning, kan man uttrykke den deriverte til sigmoid ved funksjonen selv:

$$\sigma'(x)=\sigma(x)(1-\sigma(x))$$

ReLU (rectified linear unit) er definert ved:

$$\mathrm{ReLU}(x)=max(x,0)$$

Altså $\mathrm{ReLU}(x)=x$ for alle $x>0$, og 0 ellers. Den deriverte blir:

$$\mathrm{ReLU}'(x)=\begin{cases}1 & \text{if } x>0\\ 0 & \text{if } x\leq 0\end{cases}$$

Oppgave 2.1: Implementer aktiveringsfunksjonene sigmoid og ReLU og deres deriverte i klassene gitt under. Metoden `__call__()` skal returnere et kall på funksjonene, mens metoden `diff()` skal returnere den deriverte. Inputet skal her være en to-dimensjonal array, av størrelse `[n x p]`, der `n` er antall datapunkter og `p` er antall noder i laget (som for første lag er lik antall trekk i inndataene). Outputet skal derfor også være av samme dimensjon. Som i oblig 2a kan dere velge om dere vil vektorisere eller loope eksplisitt, men her blir det nok betydelig enklere (og fortsatt mer effektivt) å vektorisere.

- a) Implementer klassen `Sigmoid`. **Hint**: Siden den deriverte til sigmoid er definert som et uttrykk av seg selv, kan dere bruke implementasjonen deres i `__call__()`, som kan kalles med `self(x_data)`.
- b) Implementer ReLU klassen. Pass spesielt på at den deriverte returnerer en NumPy-array med typen `float32` (flyttall representert med 32 bits, som er datatypen vanligvis brukt i maskinlæring). Dere kan bruke `my_array.astype(np.float32)` for dette, gitt et array `my_array`.

```
class Sigmoid:
    def __call__(self, x_data):
        """
        Performs the sigmoid function on every element in `x_data`.
        Sigmoid is defined as 1 / (1 + exp(-x)).

        Arguments:
            x_data (np.array): [n x p]-shaped array of `n` inputs and
`p` features with float values.

        Returns:
            np.array: [n x p]-shaped array of with sigmoid applied to
the elements in `x_data`.
        """
        # Sigmoid brukes for å klemme alle verdier inn i intervallen
```

```
(0, 1).
        # Dette er nyttig, på gunn av at vi ønsker ikke-lineære
transformasjoner, som
        # er glatte og deriverbare.

        # Formelen er sigmoid(x) = 1 / (1 + exp(-x))

        # Siden x_data er et numpy-array, så vil np.exp(-x_data)
beregnes
        # elementvis for hele matrisen. Dette er altså vektorisering
og er enklere
        # og mer effektivt enn eksplisitte løkker i Python.
        exponent_part = np.exp(-x_data)     # Her regner vi e^(-x) per
element
        denominator = 1 + exponent_part     # Her regner vi nevneren 1
+ e^(-x)
        sigmoid_output = 1 / denominator    # Her regener vi ut hele
brøken

        return sigmoid_output

    def diff(self, x_data):
        """
        Returns derivative of the sigmoid functions, s'(x) = s(x) * (1
- s(x))

        Arguments:
            x_data (np.array): [n x p]-shaped array of `n` inputs and
`p` features with float values.

        Returns:
            np.array: [n x p]-shaped array of the derivatives of the
sigmoid function.
        """
        # Den deriverte til sigmoid kan skrives ved å bruke sigmoid-
funksjonen slik:
        # sigmoid'(x) = sigmoid(x) * (1 - sigmoid(x))
        # På grunn av dette, så kan vi beregne sigmoid(x) først, og så
bruke verdien
        # i uttrykket for den deriverte av sigmoid(x).
        sigmoid_values = self(x_data)       # Dette er det samme som
__call__(x_data)
        derivative = sigmoid_values * (1 - sigmoid_values)

        return derivative


test_sigmoid_class(input_class=Sigmoid)
```

```python
class ReLU:
    def __call__(self, x_data):
        """
        Performs the ReLU function on every element in `x_data`.
        ReLU is defined as ReLU(x) = max(0, x).

        Arguments:
            x_data (np.array): [n x p]-shaped array of `n` inputs and
`p` features with float values.

        Returns:
            np.array: [n x p]-shaped array of the with ReLU applied to
`x_data`.
        """
        # ReLU står for Rectified Linear Unit og er definert som:
        # ReLU(x) = max(0, x)
        # Som betyr at hvis x > 0, så return x. Hvis x <= 0, return 0.
        # np.maximum sammenligner elementvis to verdier eller arrayer
og
        # returnerer det største elementet i hver posisjon.
        # Her sammenlignes hvert element i x_data med 0.
        relu_output = np.maximum(0, x_data)

        return relu_output

    def diff(self, x_data):
        """
        Returns derivative of the ReLU function, which is 1 for
elements larger than 0 and 0 if not.

        Arguments:
            x_data (np.array): [n x p]-shaped array of `n` inputs and
`p` features with float values.

        Returns:
            np.array: [n x p]-shaped array of the derivatives of the
ReLU function.
        """
        # Den deriverte til ReLU er 1 når x>0 og 0 når x<=0.
        # Sammenligningen (x_data > 0) gir oss en boolsk array med
True der
        # elementer er større enn 0, False ellers.
        # Siden oppgaven eksplisitt sier at det skal være output
verdier på float32 format,
        # så konverterer vi:
        derivative = (x_data > 0).astype(np.float32)
        return derivative


test_relu_class(input_class=ReLU)
```

Tips: Metoden `__call__` blir kalt dersom en instans av klassen blir brukt som en funksjon. Om dere for eksempel lager en instansvariabel `sigmoid = Sigmoid()`, kan dere bruke sigmoid-funksjonen med `sigmoid(x)`, som bak kulissene vil kalle `__call()__` og gi det samme resultatet som `sigmoid.__call__(x)`. For å bruke den deriverte kan man kalle `sigmoid.diff(x)`.

```
# Eksempeldata for å teste funksjonaliteten til sigmoid og ReLU
ex_array = np.array([
        [-2.0, -0.5, 0.0, 0.5, 2.0],
        [3.0, -1.0, 1.5, 0.0, -4.0]
    ])

sigmoid = Sigmoid()
relu = ReLU()

print("Input:")
print(ex_array)

print("\nSigmoid(x):")
print(sigmoid(ex_array))

print("\nSigmoid'(x):")
print(sigmoid.diff(ex_array))

print("\nReLU(x):")
print(relu(ex_array))

print("\nReLU'(x):")
print(relu.diff(ex_array))
```

```
Input:
[[-2.  -0.5  0.   0.5  2. ]
 [ 3.  -1.   1.5  0.  -4. ]]

Sigmoid(x):
[[0.11920292 0.37754067 0.5        0.62245933 0.88079708]
 [0.95257413 0.26894142 0.81757448 0.5        0.01798621]]

Sigmoid'(x):
[[0.10499359 0.23500371 0.25       0.23500371 0.10499359]
 [0.04517666 0.19661193 0.14914645 0.25       0.01766271]]

ReLU(x):
[[0.  0.  0.  0.5 2. ]
 [3.  0.  1.5 0.  0. ]]

ReLU'(x):
[[0. 0. 0. 1. 1.]
 [1. 0. 1. 0. 0.]]
```

Forklaring på oppgave 2.1 aktiveringsfunksjoner

I denne oppgaven ble aktiveringsfunksjonene Sigmoid og ReLU implementert, i tillegg til deres deriverte. Aktiveringsmetoder er en sentral del av nevrale nettverk, nettop fordi de bruker ikke-linearitet i modellene sine. Hadde vi ikke hatt denne ikke-linearitet i modellene, så ville uttrykksevnen blitt redusert.

Sigmoid

Sigmoid-funksjonen er definert som:

$$\left[\sigma'(x) = \frac{1}{1+e^{-x}}\right)$$

Denne funksjonen tar en vilkårlig reell verdi som input og returnerer en verdi mellom 0 og 1. Den brukes ofte når man ønsker en glatt og kontinuerlig transformasjon, særlig i sammenhenger der output kan tolkes som en sannsynlighet.

Den deriverte til Sigmoid er:

$$\left[\sigma'(x) = \sigma(x)(1-\sigma(x))\right)$$

Et viktig poeng her er nemlig det at den deriverte kan uttrykkes direkte ved å ta i bruk den originale sigmoid-funksjonen, som er "enkelt" og effektiv, siden vi kan begynne med å berenge

$$\sigma(x)$$

og deretter bruke denne verdien til å finne den deriverte.

ReLU

ReLU står for Rectified Linear Unit og er definert som:

$$\left[\mathrm{ReLU}(x) = max(0, x)\right)$$

Dette representerer det at ikke-positive verdier blir satt til 0, mens positive verdier ikke endres. ReLU er faktisk en av de mest brukte aktiveringsfunksjonene som blir brukt i nevrale nettverk sine skjulte lag, nettop fordi den er så enkel å beregne, og så klart fordi den fungerer godt i praksis.

Den deriverte til ReLU er gitt slik:

$$\mathrm{ReLU}'(x) = \left\{\begin{matrix} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{matrix}\right)$$

I implementasjonen så returneres den deriverte som en numpy-array med datatypen float32, som spesifisert i oppgaven. Grunnen til at float32 brukes er at det er 32 bit lagring, som er mye mindre enn float64, og for våre formål så gir float32 tilstrekkelig presisjon.

Vektorisering

Begge aktiveringsfunksjonene blir aktivert med numpy vektorisering, altså ikke python løkker, men kalkulasjoner utføres over hele arrayet med én gang. Dette gjør som sagt koden lett å lese og ikke minst svært effektiv.

Tilbakepropagering

Selv om vi i denne obligen ikke skal implementere tilbakeprogagering (backpropagation), så er det viktig å vite at under trening så brukes de deriverte i kjerneregel-beregninger. Det brukes for å finne hvordan tapet endrer seg med tanke på vektene i nettverket.

2.2 - Parametere [2p]

I de neste oppgavene skal vi bruke klassen `NeuralNetwork` til å implementere nevrale nettverk. Klassen inneholder noen metoder som er ferdig implementert og noen som skal implementeres som del av denne obligen. La oss starte med å opprette en instans av klassen. Konstruktøren tar inn to argumenter, en liste med antall noder i hvert lag og en liste med tilsvarende aktiveringsfunksjoner.

```
neural_network = NeuralNetwork(
    layer_sizes=[784, 32, 16, 10],
    activation_functions=[ReLU(), ReLU(), IdentityActivation()],
)
```

Her lager vi et nevralt nettverk med fire lag: Et input-lag med 784 noder, som tilsvarer oppløsningen i MNIST på 28 x 28 piksler, et skjult lag med 32 noder, et til skjult lag med 16 noder, og til slutt et output-lag med 10 noder, som tilsvarer antall klasser i MNIST.

Lagene trenger en aktiveringsfunksjon for å regne ut aktiveringene i hvert lag, bortsett input-laget (siden aktiveringene i det første laget er lik inputet). Disse funksjonene må være implementert som en klasse, slik vi gjorde i forrige seksjon, med `__call__()` som kaller funksjonen og `diff()` som kaller dens deriverte. Vi bruker `IdentityActivation()` i siste lag, som returnerer det samme som den får som input, siden aktiveringsfunksjonen for output-laget `softmax` blir brukt implisitt i bakoverfasen og som del av evalueringsfunksjonen vår senere.

Konstruktøren til klassen er ferdiglaget og initialiserer parameterne. I lineære modeller initialiseres ofte parameterne til 0-ere, men trening av dype nevrale nettverk er litt mer intrikat og dette kan føre til dårlig trening. Derfor initialiseres vektene til *små tilfeldige positive og negative verdier*.

I tillegg setter konstruktøren noen klassevariabler dere vil trenge senere, som `self.layer_sizes` (samme som argumentet til konstruktøren med samme navn), `self.n_layers` (antall lag, lengden av `self.layer_sizes`) og `self.activation_functions` (samme som argumentet til konstruktøren med samme navn). I tillegg blir vektene og konstantleddene lagret i henholdsvis klassevariablene `self.weights` og `self.biases`. Disse variablene er lister, der element nummer i tilsvarer vektene eller konstantleddene for lag nummer i.

I neste oppgave skal vi bli litt mer kjent med formatet til vektene og konstantleddene.

Oppgave 2.2: Lag en metode `count_parameters()` som returnerer antall parametere (vekter pluss konstantledd) i modellen.

Hint: Dette krever å iterere over `self.weights` og `self.biases`. Hvert element i `self.weights` er en to-dimensjonal NumPy array. Dimensjonene i lag i kan nås ved `self.weights[i].shape` og tilsvarende for konstantleddene.

Hint: For antall konstantledd, bruk `self.biases[i].shape[1]`. For antall vekter, gang nulte og første dimensjon sammen.

```
def count_parameters(self):
    """
    Counts the amount of trainable parameters in the model. This is
the sum of nodes in each layers (weights) plus the biases.

    Returns:
        int: The amount of trainable parameters in the model.
    """
    total_parameters = 0

    # Vi går her lag for lag og teller parameterne
    for i in range(len(self.weights)):
        # Leser ut dimensjonene til vektmatrisen i lag "i":
        weight_shape = self.weights[i].shape
        num_rows = weight_shape[0]      # Antall inputkoblinger inn i
laget
        num_cols = weight_shape[1]      # Antall noder i laget

        # Antall vekter = antall rader * antall kolonner
        num_weights = num_rows * num_cols

        # Bias shape er vanligvis (1, ant_noder_i_lag)
        bias_shape = self.biases[i].shape
        num_biases = bias_shape[1]

        # Totalt for dette laget
        layer_total = num_weights + num_biases

        # Legger til i totalsummen
        total_parameters += layer_total

    return total_parameters


NeuralNetwork.count_parameters = count_parameters
test_count_parameters(input_class=NeuralNetwork)

# Nevral nettverk opprettelse og testing
neural_network = NeuralNetwork(
```

```
    layer_sizes=[784, 32, 16, 10],
    activation_functions=[ReLU(), ReLU(), IdentityActivation()],
)

# Skriver ut shapes for å forstå parameterene i lagene, lag for lag
for i in range(len(neural_network.weights)):
    print(f"Lag {i+1}:")
    print(f"    weights.shape = {neural_network.weights[i].shape}:")
    print(f"    biases.shape = {neural_network.biases[i].shape}:")

    num_weights = neural_network.weights[i].shape[0] *
neural_network.weights[i].shape[1]
    num_biases = neural_network.biases[i].shape[1]

    print(f"    antall vekter = {num_weights}")
    print(f"    antall biaser = {num_biases}")
    print(f"    totalt i laget = {num_weights + num_biases}\n")

print("Totalt antall parametere i modellen:",
neural_network.count_parameters())
```

```
Lag 1:
    weights.shape = (32, 784):
    biases.shape = (1, 32):
    antall vekter = 25088
    antall biaser = 32
    totalt i laget = 25120

Lag 2:
    weights.shape = (16, 32):
    biases.shape = (1, 16):
    antall vekter = 512
    antall biaser = 16
    totalt i laget = 528

Lag 3:
    weights.shape = (10, 16):
    biases.shape = (1, 10):
    antall vekter = 160
    antall biaser = 10
    totalt i laget = 170

Totalt antall parametere i modellen: 25818
```

Forklaring av resultatene

Resultatene viser at modellen har tre sett med trenbare parametere, siden input-laget ikke har egne vekter eller biaser. For hvert av de tre neste lagene teller vi både antall vekter og antall biaser.

I lag 1 er weights.shape = (32, 784) og biases.shape = (1, 32). Dette betyr at laget har 32 noder, og at hver node er koblet til 784 inputverdier. Dermed blir antall vekter 32*784 = 25088, og med 32 biaser får laget totalt 25120 paramatere.

I lag 2 er weights.shape = (16, 32) og biases.shape = (1, 16). her har laget 16 noder som hver mottar input fra 32 noder i forrige lag. Det gir 16*32 = 512 vekter og 16 biaser, altså 528 totale parametere.

I lag 3 er weights.shape = (10, 16) og biases.shape = (1, 10). Dette er output-laget med 10 noder, der hver node mottar input fra 16 noder i forrige lag. Det gir 10*16 = 160 vekter og 10 biaser, altså 170 totale parametere.

Når vi så summerer alle lagene, får vi: 25120 + 528 + 170 = 25818

Modellen har altså totalt 25818 trenbare parametere. Dette stemmer med utskriften fra funksjonen og viser at count_paramterers() fungerer som forventet.

2.3 - Foroverfasen (forward pass) [6p]

Vi skal nå implementere foroverfasen (forward pass). Dette steget sender et input gjennom alle lagene i nettverket. For MNIST datasettet vil inputet være en liste av lengde 784, som representerer et svart-hvitt bilde av oppløsning 28 x 28, og outputet vil være en liste av lengde 10, som representerer et tall for hver av de 10 mulige sifrene. Når vi bruker modellen til klassifisering vil vi velge sifferet tilsvarende den av de 10 output-nodene som har høyest verdi. Foroverfasen blir også brukt under trening, der vi altererer mellom foroverfasen og bakoverfasen.

La oss se nærmere på hva som skjer i foroverfasen. I hvert lag ganger vi aktiveringene av nodene i det forrige laget med vektene fra det neste laget, plusser på konstantleddet (biasen) og sender dette gjennom en aktiveringsfunksjon. I utgangspunktet er dette ganske enkle operasjoner, men man må konsentrere seg litt for få indeksene til vektene og lagene korrekt. La oss kort gjennomgå presis notasjon for dette. En nøyere gjennomgang er gitt i oppgave 7.2 i ukesoppgaver for uke 7.

Vi starter med notasjonen av vektene våre. Når vi går fra et lag l til det neste laget $l+1$, har vi én vekt mellom hver node i lag l og hver node i lag $l+1$. Disse vektene blir representert i en *matrise*, altså en todimensjonal tabell, som vi kaller for $W^{(l+1)}$. Her er bruker vi $(l+1)$ til å representere at dette er vektene for lag nummer $l+1$, så $l+1$ er altså *ikke* en eksponent. Elementet $w_{j,k}^{(l+1)}$ vil være vekten i rad j og kolonne k i denne matrisen. I notasjonen vår vil dette representere vekten fra node nummer k i lag nummer l til node j i lag nummer $l+1$. Dette blir illustrert med noen eksempler i figuren under.

Illustrasjonen viser et eksempel på et enkelt nevralt nettverk med fire noder i inputlaget, to noder i det eneste skjulte laget (lag 2) og tre noder i outputlaget. Vi får to vekt-matriser, $W^{(2)}$ og $W^{(3)}$. Disse kan vi notere som:

$$W^{(2)}=\begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} & w_{1,3}^{(2)} & w_{1,4}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} & w_{2,3}^{(2)} & w_{2,4}^{(2)} \end{bmatrix} W^{(3)}=\begin{bmatrix} w_{1,1}^{(3)} & w_{1,2}^{(3)} \\ w_{2,1}^{(3)} & w_{2,2}^{(3)} \\ w_{3,1}^{(3)} & w_{3,2}^{(3)} \end{bmatrix}$$

Merk at at vekt $w_{j,k}^{(l)}$ går fra node k i lag $l-1$ til node j i lag l. Dette kan virke litt omvendt, men det gjør at vektor- og matrisenotasjonen blir enklere. To vekter er notert i illustrasjonen som eksempel.

I illustrasjonene for lineær og logistisk regresjon var konstantleddet tegnet som en egen node, men for nevrale nettverk er de som regel implisitte fra illustrasjonene. Hvert lag, bortsett fra input-laget, har en vektor med konstantledd med lengde lik antall noder i laget. Vi bruker nå *kolonnevektorer* for at vektor- og matrisenotasjonen skal bli korrekt. Konstantleddene i eksemplet over blir altså:

$$b^{(2)}=\begin{bmatrix} b_1^{(2)} \\ b_2^{(2)} \end{bmatrix} b^{(3)}=\begin{bmatrix} b_1^{(3)} \\ b_2^{(3)} \\ b_3^{(3)} \end{bmatrix}$$

der $b_j^{(l)}$ tilsvarer konstantleddet for node j i lag l.

La oss se på hvordan vi regner oss fremover i nettverket. Vi regner ut aktiveringene for nodene i et lag ved å bruke aktiveringene for forrige lag, der aktiveringene for første lag er lik inputet. Først ganger vi aktiveringene fra forrige lag sammen med tilsvarende vekter og plusser på tilsvarende konstantledd. Dette gir oss den *vektede summen* for dette laget, som vi bruker bokstaven z til å representere. Vi får aktiveringene, som vi bruker bokstaven a til å representere, ved å sende den vektede summen gjennom aktiveringsfunksjonen for laget. La oss se på spesifikt hvordan vi kan regne ut aktivering for node 1 i lag 2 i eksemplet vårt:

For å regne ut dette får vi:

$$a_1^{(2)}=f_2\left(z_1^{(2)}\right)=f_2\left(\sum_{k=1}^{4} w_{1,k}^{(2)} x_i+b_1^{(2)}\right)$$

$$¿ f_2\left(w_{1,1}^{(2)} x_1+w_{1,2}^{(2)} x_2+w_{1,3}^{(2)} x_3+w_{1,4}^{(2)} x_4+b_1^{(2)}\right)$$

som altså tilsvarer å gange aktiveringene fra forrige lag (inputene) med de tilsvarende vektene (markert med fet pil i illustrasjonen), plusse de resulterende produktene sammen i tillegg til å plusse på det tilsvarende konstantledd, før vi sender det gjennom f_2, som er aktiveringsfunksjonen for lag 2. Vi kan regne ut $a_2^{(2)}$ tilsvarende ved å bytte ut hvilke vekter og konstantledd vi bruker.

Det samme konseptet gjelder for lag 3. For aktiveringen til andre node i tredje lag kan vi vise dette grafisk ved følgende illustrasjon:

Utregningen kan vi notere ved følgende:

$$a_2^{(3)}=f_3\left(z_2^{(3)}\right)=f_3\left(\sum_{j=1}^{2} w_{2,k}^{(3)} a_k^{(2)}+b_2^{(3)}\right)=f_3\left(w_{2,1}^{(3)} a_1^{(2)}+w_{2,2}^{(3)} a_2^{(2)}+b_2^{(3)}\right)$$

Det samme konseptet gjelder som før: For å få aktiveringene for neste lag, gang aktiveringene fra forrige lag sammen med tilsvarende vekter, pluss dem sammen, pluss på tilsvarende konstantledd og send det gjennom aktiveringsfunksjonen.

Generelt sett får vi at aktiveringen for node j i lag l kan regnes ut ved:

$$a_j^{(l)} = f_l\left(z_j^{(l)}\right) = f_l\left(\sum_{k=1}^{n_{l-1}} w_{j,k}^{(l)} a_k^{l-1} + b_j^{(l)}\right)$$

der n_{l-1} er antall noder i lag $l-1$.

Når dette implementeres kan vi bruke en generell loop som sjekker dimensjonene til vektene, slik at vi kan lage én implementasjon som fungerer for alle mulige antall av lag og noder i lagene.

Vektor og matrisenotasjon: Det er vanlig å notere dette med vektorer og matriser. Det gir mer kompakt notasjon uten summetegn og indekser, som dermed kan bli enklere å lese om man venner seg til denne måten å notere på. Denne notasjonen likner også på vektorisert kode, som gir mer effektiv implementasjon. For at koden ikke skal bli altfor treig burde multiplikasjonen og addisjonen av vektene og konstantleddene vektoriseres. La oss uansett gå igjennom hvordan notasjonen blir så vi kan vende oss til den.

Vi kan uttrykke alle aktiveringene for et lag med én formel:

$$a^{(l)} = f_l\left(z^{(l)}\right) = f_l\left(W^{(l)} a^{(l-1)} + b^{(l)}\right)$$

Her er $W^{(l)} a^{(l-1)}$ multiplikasjon mellom matrisen $W^{(l)}$ og vektoren $a^{(l-1)}$, som gir oss akkurat den multipliseringen og addisjonen vi er ute etter.

La oss se på dette i eksemplet vårt. For aktiveringen i andre lag har vi to noder, og vi har fire noder i det forrige laget. Vi får:

$$a^{(2)} = f_2\left(z^{(2)}\right) = f_2\left(W^{(2)} x + b^{(2)}\right)$$

der

$$W^{(2)} x + b^{(2)} = \begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} & w_{1,3}^{(2)} & w_{1,4}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} & w_{2,3}^{(2)} & w_{2,4}^{(2)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b_1^{(2)} \\ b_2^{(2)} \end{bmatrix}$$

$$¿ \begin{bmatrix} w_{1,1}^{(2)} x_1 + w_{1,2}^{(2)} x_2 + w_{1,3}^{(2)} x_3 + w_{1,4}^{(2)} x_4 + b_1^{(2)} \\ w_{2,1}^{(2)} x_1 + w_{2,2}^{(2)} x_2 + w_{2,3}^{(2)} x_3 + w_{2,4}^{(2)} x_4 + b_1^{(2)} \end{bmatrix}$$

Tips: Oppgave 7.2 i ukesoppgaver for uke 7 går igjennom notasjonen i nærmere detalj sammen med eksempler på utregning.

Merk: Det finnes dessverre mange forskjellige notasjoner for nevrale nettverk. Vi bruker notasjonen fra boken Neural Networks and Deep Learning, som er samme notasjon brukt i videoene til 3Blue1Brown. Dette er en moderne og mye brukt notasjon, men siden konvensjonene varierer kommer det til å være annerledes i andre kontekster.

Med dette har vi vært igjennom notasjonen som kreves for foroverfasen i nevrale nettverk. La oss implementere det.

Oppgave 2.3: Implementer foroverfasen for klassen `NeuralNetwork`. Merk at metodene under forventer input `x_data` som en array med dimensjonene `[n x m]`, der n er antall datapunkter og m er antall noder i det forrige laget.

Vektorisering: Det blir flere løkker i utregningen. La oss se på dem. Først itereres det over antall datapunkter. Her blir det enklest å iterere eksplisitt, men det er også mulig å vektorisere denne delen til slutt for de som har lyst på en ekstra utfordring. Det itereres også over lagene i nettverket, dette er ikke mulig å vektorisere. Til slutt må selve utregningen av aktiveringene itereres over. Dette tilsvarer to løkker: en for antall noder i neste lag (for å regne ut alle aktiveringene) og en for antall noder i forrige lag (for å gange med tilsvarende vekter). Denne koden må vektoriseres for at treningen ikke skal ta for lang tid. Dette vil også gi mer kompakt kode med færre linjer og indekseringer.

Hint: Bruk `@` for matrise-vektor-multiplikasjon med NumPy-arrays. Prøv å vektorisere $W^{(l)}a^{(l-1)}+b^{(l)}$ slik. Konstantleddet kan plusses direkte på (NumPy håndterer dimensjonene med *broadcasting*).

- a) Implementer metoden `forward()`. Denne metoden gjennomfører foroverfasen og returnerer verdiene i det siste laget, som ofte er kalt $logits$. Implementasjonen må fungere for et vilkårlig antall lag, så metoden må iterere over `range(self.n_layers - 1)`. De vektede summene og aktiveringene for hvert lag fra foroverfasen blir også brukt i bakoverfasen, så dere må lagre dem i klassevariablene `self.weighted_sums` og `self.activations`, representert med en liste der element nummer i er aktiveringene for lag nummer i (aktiveringene for det første laget tilsvarer inputet). Dette er hoveddelen av denne obligen, så vær forberedt på å bruke litt tid på denne oppgaven.
- b) Implementer metoden `predict()` som gitt et input returnerer prediksjonen til det nevrale nettverket. Denne metoden skal kalle på `forward()`, som gir aktiveringene til det siste laget, og returnere den klassen som har høyest aktivering (for hver node). Funksjonen `np.argmax()` kan være nyttig til dette.

```
def forward(self, x_data):
    """
    Feeds the data forward. Do not discretize the output (give logits
values).
    For each layer, calculates the activations, and feeds forward.

    Arguments:
        x_data (np.array): [n x m] data to forward.

    Returns:
        activations (np.array): [n x n_c] array over logits outputs
(activations of last layer).
    """
    # Disse listene lagrer mellomresultater fra foroverfasen.
    # De brukes i bakoverfasen når gradienetene skal beregnes.
    self.weighted_sums = []     # Lagrer z-verdier for hvert lag
    self.activations = []       # Lagrer a-verdier for hvert lag
```

```
    # Aktiveringene i første lag er inputdataene selv.
    # Dette tilsvarer a^(1) i notasjonen.
    activations = x_data

    # Lagrer inputet som første element i self.activations,
    # siden oppgaven sier at aktiveringene for første lag er inputet.
    self.activations.append(activations)

    # Iterer gjennom alle lagovergangene i nettverket. Hvis vi har
n_layers
    # lag totalt, så har vi n_layers - 1 sett med vekter og biaser
mellom lagene.
    for i in range(self.n_layers - 1):
        # Vektmatrisene er laget med shape: (ant noder i nytt lag, ant
noder i forrige lag)
        # Aktiveringene har shape: (ant datapunkter, ant noder i
forrige lag)
        # Bruker weights.T for å få matriseproduktene for alle
datapunktene samtidig
        # activations @ weights.T
        # Ved å bruke dette får vi: (ant datapunkter, ant noder i nytt
lag)
        # Til slutt legger vi til bias som har shape (1, ant noder i
nytt lag), og
        # numpy bruker broadcasting slik at bias legges inn for hver
rad
        weighted_sum = activations @ self.weights[i].T +
self.biases[i]

        # Lagrer den vektede summen z^(1) fordi den trengs senere i
        # bakoverfasen under gradientberegningen.
        self.weighted_sums.append(weighted_sum)

        # Nå er vi på steget, der vi sender den vektede summen over
        # aktiveringsfunksjonen til laget for å få nye aktiveringer.
        # self.activation_funciton[i] er en instans av ReLU(),
Sigmoid() eller
        # IdentityActivation(), og kalles som en funksjon
        activations = self.activation_functions[i](weighted_sum)

        # Lagrer aktiveringene for laget her.
        self.activations.append(activations)

    # Etter siste iterasjon, så inneholder activations output fra
siste lag (logits)
    return activations
```

```
NeuralNetwork.forward = forward
test_forward_pass(input_class=NeuralNetwork, relu_class=ReLU)

def predict(self, x_data):
    """
    Predicts on data, outputs the classes predicted.

    Arguments:
        x_data (np.array): [n x p]-shaped data to predict on.

    Returns:
        preds (np.array): [n]-shaped array of predicted classes (not
one-hot-encoded).
    """
    # Først kjører vi en full foroverfase.
    # Dette gir output fra siste lag, altså ett sett med logits per
datapunkt
    logits = self.forward(x_data)

    # Hver rad i logits er ett datapunkt og hver kolonne er en klasse.
    # Klassen med høyest verdi er prediksjonen.
    # axis=1 brukes for å finne max langs kolonnene for hver rad.
    predictions = np.argmax(logits, axis=1)

    return predictions


NeuralNetwork.predict = predict
test_predict(input_class=NeuralNetwork, relu_class=ReLU)

# Test celle for oppgave 2.3
# Lager et nytt nevralt nettverk
neural_network = NeuralNetwork(
    layer_sizes=[784, 32, 16, 10],
    activation_functions=[ReLU(), ReLU(), IdentityActivation()],
)

# Bruker noen få treningsbilder som eksempel
ex_example = x_train[:3]

# Kjører foroverfasen
logits = neural_network.forward(ex_example)

print("Input shape:", ex_example.shape)
print("")

for i in range(len(neural_network.weighted_sums)):
    print(f"Lag {i+1} -> {i+2}")
    print(f"    weights.shape           =
{neural_network.weights[i].shape}")
    print(f"    biases.shape            =
```

```
{neural_network.biases[i].shape}")
    print(f"    weighted_sums.shape     =
{neural_network.weighted_sums[i].shape}")
    print(f"    activations.shape       =
{neural_network.activations[i+1].shape}")
    print("")

print("Logits shape:", logits.shape)
print("Prediksjoner:", neural_network.predict(ex_example))
```

```
Input shape: (3, 784)

Lag 1 -> 2
    weights.shape           = (32, 784)
    biases.shape            = (1, 32)
    weighted_sums.shape     = (3, 32)
    activations.shape       = (3, 32)

Lag 2 -> 3
    weights.shape           = (16, 32)
    biases.shape            = (1, 16)
    weighted_sums.shape     = (3, 16)
    activations.shape       = (3, 16)

Lag 3 -> 4
    weights.shape           = (10, 16)
    biases.shape            = (1, 10)
    weighted_sums.shape     = (3, 10)
    activations.shape       = (3, 10)

Logits shape: (3, 10)
Prediksjoner: [9 9 9]
```

Forklaring oppgave 2.3

I denne oppgaven implementerte jeg foroverfasen til nettverket. Foroverfasen sender inputdata gjennom alle lagene i modellen og i hvert lag beregnes først en vektet sum og deretter nye aktiveringer ved hjelp av lagets aktiveringsfunksjon.

I testen jeg laget, så hadde inputdataene shape (3, 784), noe som betyr at jeg sendte inn 3 bilder samtidig, der hvert bilde har en pikselverdi på 784. Etter første lag fikk jeg weighted_sums.shape = (3, 32) og activations.shape = (3, 32), som viser oss at hver bilde ble transformert til 32 aktiveringer. Deretter gikk disse verdiene til et lag med 16 noder, som ga shape (3, 16), og til slutt output-laget med 10 noder, som ga logits.shape = (3, 10).

Siden vektene blir laget med shape (ant noder i nytt lag, ant noder i forrige lag), så måtte jeg bruke self.weights[i].T i multiplikasjonen. På grunn av dette, ble den vektede summen activations @ self.weights[i].T + self.biases[i].

Metoden forward() returnerer altså logits fra siste lag, mens predict() velger klassen med høyest outputverdi med np.argmax(). I testen over ble prediksjonene [9, 9, 9].

2.4 - Trening [1p]

Metodene for bakoverfasen og trening er allerede implementert, så med foroverfasen implementert er klassen for nevrale nettverk nå klar til å trenes. Selv om det ikke er pensum å kjenne til den spesifikke utregningen for bakoverfasen, er det viktig (og pensum) å forstå hva den gjør. La oss derfor se litt på dette.

Tilbakepropagering (backpropagation)

Vi vil bruke gradientnedstigning (som vi straks skal generalisere til *mini-batch stochastic gradient descent* (SGD)) til å iterativt oppdatere vektene våre slik at tapsfunksjonen vår blir minimert. For å gjøre dette trenger vi å finne *gradienten til tapsfunksjonen*. Dette er altså samlingen av tapsfunksjonen *derivert med hensyn på hver av vektene og konstantleddene*. Disse deriverte sier altså "*dersom denne vekten endres litt, hvor mye endrer tapsfunksjonen seg?*". Deretter oppdaterer vi vektene i retning av den negative gradienten, som altså tilsvarer den retningen vi må endre vektene og konstantleddene for å minske tapsfunksjonen mest mulig.

For å klare å gjøre dette trenger vi en måte å finne gradienten på. Heldigvis er derivasjon en relativt enkel prosess, og selv om det kan bli knotete å regne ut den deriverte for hånd finnes det spesifikke regler som gjør det enkelt for datamaskiner. Gitt et uttrykk for den deriverte av tapsfunksjonen og aktiveringsfunksjonene våre kan vi bruke *kjerneregelen* (chain rule) og reglene for derivasjon av multiplikasjon til å finne dette.

Vi trenger allikevel et triks til. Nevrale nettverk har ofte millioner (eller til og med milliarder) av parametere, og dersom vi skulle ha regnet ut den deriverte med hensyn på vektene hver for seg, hadde dette blitt en enormt krevende prosess. Her kommer algoritmen *tilbakepropagering* inn. Tilbakepropagering finner gradienten effektivt som en rekursiv prosess. Først regner den ut aktiveringen for hvert av lagene i foroverfasen. Deretter starter den i det bakerste laget og regner ut gradienten til vektene i laget én og én bakover i nettverket. Trikset består av å *uttrykke gradienten for vektene i lag* l *som en funksjon av gradienten til vektene i lag* $l+1$. Dette reduserer antall nødvendige beregninger betydelig.

Stokastisk gradientnedstigning (Stochastic Gradient Descent (SGD))

Det gjenstår et viktig triks for å gjøre treningen for nevrale nettverk, som er kalt for *mini-batch stokastisk gradientnedstigning*. Dyplæring krever ofte store mengder data. I MNIST har vi 60000 treningsbilder (eller 50000 treningsbilder og 10000 valideringsbilder), som allerede er mer enn mange maskinlæringskontekster, men det er fortsatt relativt lite for dyplæring. Bildemodeller blir ofte trent opp på millioner (og til og med milliarder) av bilder, og store språkmodeller kan bli trent på omtrent all tekst på hele internett. Dette gjør at man ikke kan, eller at det ikke lønner seg å, regne ut gradienten for alle datapunktene i treningsettet på en gang.

Enklere lineære modeller oppdaterer vanligvis vektene med hensyn på gradienten til alle datapunktene i datasettet av gangen, mens vi gjerne velger en delmengde (en "mini-batch") for nevrale nettverk. Dersom vi hadde brukt alle dataene for nevrale nettverk ville vi møtt mange problemer. Til å starte med kan treningen gå saktere, siden vi må regne ut gradienten til titusenvis eller millioner av datapunkter før vi får oppdatert vektene noe som helst. Dette blir ofte ikke en gang mulig, siden datasettene er ofte så store at de ikke vil ha plass i minne på datamaskinen. Videre får man i praksis ofte bedre resultater med å ikke bruke gradientene til

alle datapunktene av gangen, som stammer fra at loss-landskapet (tapet som funksjon av vektene og konstantleddene) er komplekst og endrer seg fort for nevrale nettverk.

Løsningen er altså å bruke færre datapunkter for hver vektoppdatering. I den originale stokastiske gradientnedstigningen itererer vi over ett og ett datapunkt, regner ut gradienten for det, og oppdaterer deretter alle parameterne med hensyn på dette ene datapunktet. Når alle datapunktene er iterert over har man fullført en epoke. Dette er vanligvis noe ekstremt, så man bruker vanligvis en delmengde av datapunkter (antallet kan variere mye avhengig av konteksten). Dette gir oss *mini-batch stokastisk gradientnedstigning*. Vi setter altså en mini-batch-størrelse b, trekker b antall datapunkter fra datasettet vårt (uten tilbakelegging), regner ut gradienten for disse datapunktene og oppdaterer vektene med hensyn på denne gradienten. Når vi har gjort dette for alle datapunktene har vi fullført en epoke.

Mini-batch stokastisk gradientnedstigning gir oss flere fordeler over normal stokastisk gradientnedstigning (som bare tar ett datapunkt av gangen). For det første kan treningen skje mer effektivt, siden man kan vektorisere koden som itererer over datapunktene i mini-batchen. Man får også noe mer stabil trening, siden gradienten for et og et datapunkt kan variere stort. Den viktigste grunnen er rett og slett at det pleier å fungere bedre i praksis.

Mini-batch stokastisk gradientnedstigning er så populært at det ofte blir referert til som kun stokastisk gradientnedstigning, som kan være noe forvirrende. Originalt er altså vanlig gradientnedstigning gjort på hele datasettet av gangen, stokastisk gradientnedstigning er gjort på ett og ett datapunkt, mens mini-batch stokastisk gradientnedstigning er gjort på en delmengde. Ofte blir navnet stokastisk gradientnedstigning (og forkortelsen SGD) allikevel brukt til å beskrive mini-batch stokastisk gradientnedstigning, siden det i praksis nesten alltid er den metoden man bruker.

Mer om trening

Trening av nevrale nettverk kan i praksis være mye vanskeligere enn andre maskinlæringsmodeller, som betyr at det krever mer testing av hyperparametere og flere triks under treningen for å få gode resultater. Det tok flere tiår fra flerlags nevrale nettverk og tilbakepropagering ble utviklet til det ble populært. Det er mange grunner til dette. En av dem var at det er vanskelig å trene dype nevrale nettverk. Når de bakerste lagene oppdateres vil dette endre hva vektene i de tidligere lagene må være. Det gjør det vanskelig å finne hyperparametere (læringsrate, størrelse på mini-batch, antall lag og mer) som passer. Enkle datasett som MNIST er mye enklere å trene på enn mer realistiske dataset, som krevde flere triks for å få konsistent trening (som *dropout*, *ReLU*, *Batch-norm*, andre versjoner av SGD og bedre vektinitialisering). Trening av dype nevrale nettverk krever ofte at man nøye følger med på metrikker under trening og justerer hyperparameterne deretter.

Oppgave 2.4: Beskriv kort (3-5 setninger) hvordan treningen av nevrale nettverk likner trening av lineær og logistisk regresjon og hvordan det er forskjellig.

Forklaring oppgave 2.4

Trening av nevrale nettverk ligner på trening av lineær og logistisk regresjon ved at alle modellene prøver å minimere en tapsfunksjon ved hjelp av gradientbasert oppdatering av parametere. I alle tilfellene brukes den deriverte av tapsfunksjonen for å finne hvordan vektene bør endres for å redusere feilen. Forskjellen er at nevrale nettverk har flere lag og ikke-lineære

aktiveringsfunksjoner, som gjør modellen mer uttrykksfull og vanskeligere å trene. På grunn av dette bruker nevrale nettverk tilbakepropagering for å beregne gradienter effektivt og trener ofte med mini-batch SGD i stedet for å bruke hele datasettet i hver oppdatering.

2.5 - Evaluering [2p]

Det eneste vi mangler å implementere er metrikkene våre, som er tapsfunksjonen *kryssentropi* for flere klasser og nøyaktighet (accuracy). Under trening vil disse funksjonene bli brukt på trenings- og valideringsdataene etter hver epoke og printe ut resultatet fortløpende. På denne måten kan vi holde oversikt over treningen underveis.

Multiklasse-kryssentropi er en generalisering av binær kryssentropi (BCE) som vi brukte for logistisk regresjon i oblig 2a. Til dette bruker vi *logits*-verdiene fra foroverfasen, altså verdiene vi får fra `forward()`. Disse verdiene må sendes gjennom funksjonen `softmax()` (som er ferdig implementert), som for vårt datasett gir verdier $(\hat{y}_1, \ldots, \hat{y}_{10})$, der hver $\hat{y}_i$ er mellom 0 og 1, og alle $\hat{y}_i$-ene summerer til 1. Dette sammenliknes med *one-hot-enkodingen* av de sanne verdiene, som gir verdier $\text{one-hot}(y) = (y_1, \ldots, y_{10})$ der nøyaktig én y_i er 1 og resten 0. Verdien som er 1 tilsvarer den sanne klassen. For eksempel, dersom den sanne verdien er sifferet 7, altså $y = 7$, vil $\text{one-hot}(y) = (0, 0, 0, 0, 0, 0, 0, 1, 0, 0)$. Dere kan bruke funksjonen `integer_hot_hot_encode()` til dette, som er ferdig implementert.

Tapsfunksjonen baserer seg på dette leddet:

$$-y_i \log(\hat{y}_i)$$

Dette vil vil være 0 for alle verdier i der $y_i = 0$, så det er kun ikke-null for verdien av i (den sanne verdien) vi trenger å beregne. Da blir tapet $-\log(\hat{y}_i)$, som går mot null når $\hat{y}_i$ nærmer seg 1, og gå mot uendelig når $\hat{y}_i$ nærmer seg null. For å få kryssentropien summerer vi resultatet for alle klassene i C og får:

$$\text{cross-entropy}(y, \hat{y}) = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

Det siste som gjenstår er å summere over alle datapunktene N og ta gjennomsnittet, altså dele på N.

For nøyaktigheten (accuracy) er prosessen tilsvarende tidligere. Vi bruker heltall-prediksjonene fra modellen vår, altså de vi får fra `predict()`, sammenlikner dem med de sanne verdiene og tar gjennomsnittet.

Tips: Multiklasse kryssentropi funksjonen er nøyere gjennomgått i oppgave 7.3 i ukesoppgavene for uke 7.

Oppgave 2.5: Implementer metrikkene for modellen vår.

- a) Implementer funksjonen `calculate_multiclass_cross_entropy_with_logits()` som beskrevet over. Dere kan bruke de ferdigimplementerte funksjonene `softmax()` og `integer_one_hot_encode()`. Det er flere mulige måter å implementere funksjonen på, noen bruker ikke one-hot-enkodingen.

- b) Implementer funksjonen `calculate_accuracy()`.

```
def calculate_multiclass_cross_entropy(true_values, predictions):
    """
    Computes multiclass cross entropy loss from logits.
    Performs softmax on the predictions.

    Arguments:
        true_values (np.array): [n]-shaped array of integer target
values (not one-hot encoded).
        logits (np.array): [n x n_c]-shaped array of raw model outputs
(not softmaxed).

    Returns:
        float: The cross entropy loss.
    """
    # Argumentet predictions er logits, altså output verdier fra
nettverket.
    # Disse er ikke sannsynligheter enda, så vi må først sende dem
gjennom softmax
    # funksjonen for å få verdier mellom 0 og 1 som summerer til 1 per
datapunkt
    probabilities = softmax(predictions)

    # De sanne klassene er representert som heltall og for å kunne
bruke formelen for
    # multiklasse kryssentropi, så må vi gjøre disse om til one-hot-
enkoding, der
    # riktig klasse får verdi 1 og alle andre får verdi 0.
    true_one_hot = integer_one_hot_encode(true_values)

    # Kryssentropi for ett datapunkt er: sum(y_i * log(y_hat_i)).
    # Når y er one_hot_encoded, så vil kun ett ledd i summen være lik
0, nemlig
    # leddet som tilsvarer den gitte klassen.
    log_probabilities = np.log(probabilities)

    # Nå regner vi ut tapet for hvert datapunkt ved å summere
klassene.
    # axis=1 brukes for å summere bortover kolonnene
    loss_per_datapoint = -np.sum(true_one_hot * log_probabilities,
axis=1)

    # Til slutt beregnes gjennomsnittet over datapunktene
    mean_loss = np.mean(loss_per_datapoint)
    return mean_loss


test_calculate_multiclass_cross_entropy_loss(input_function=calculate_
multiclass_cross_entropy)
```

```
Failed: `test_calculate_multiclass_cross_entropy_loss`. Test number
`1` got unexpected error: `operands could not be broadcast together
with shapes (2,2) (2,3) `.
```

```python
def calculate_accuracy(true_values, classifications):
    """
    Calculates the accuracy of predictions and labels
    This is defined as (number of correct guesses) / (number of
guesses).

    Arguments:
        true_values (np.array): [n]-shaped array of true targets for
the same inputs as the predictions.
        classifications (np.array): [n]-shaped array of predictions.

    Returns:
        float: The calculated accuracy.
    """
    # Sammenligner hver predikerte klasse med en sann klasse, som gir
oss en
    # boolsk array der True for riktig prediksjon, False ellers.
    correct_predictions = (true_values == classifications)

    # For gjennomsnittet av en boolsk array i numpy, så tolkes True=1
og False=0.
    # Dermed får vi andelen riktige prediksjoner direkte her
    accuracy = np.mean(correct_predictions)

    return accuracy


test_calculate_accuracy(input_function=calculate_accuracy)

# Test eksempel for å forstå multiklasse kryssentropi og accuracy for
oppgave 2.5
ex_true = np.array([2, 0, 1])

# Setter tre datapunkter og tre klasser, som er logits og ikke
sannsynligheter
ex_logits = np.array([
    [0.1, 0.2, 2.5], # Klasse 2
    [3.0, 0.1, 0.2], # Klasse 0
    [0.3, 1.8, 0.1] # Klasse 1
])

ex_preds = np.argmax(ex_logits, axis=1)

print("Sanne klasser:", ex_true)
print("Predikerte klasser:", ex_preds)
print("Accuracy:", calculate_accuracy(ex_true, ex_preds))
```

```
print("Cross entropy:", calculate_multiclass_cross_entropy(ex_true, ex_logits))
```

```
Sanne klasser: [2 0 1]
Predikerte klasser: [2 0 1]
Accuracy: 1.0
Cross entropy: 0.2083305141939555
```

Forklaring oppgave 2.5

I oppgave 2.5 implementerte jeg to evalueringsmål, nemlig accuracy og multiklasse kryssentropi. I test-eksempelet mitt over, er de sanne klassene [2, 0, 1] og modellen predikerte også [2, 0, 1], så accuracy blir 1.0 i dette tilfellet. Det vil altså si at prediksjonene er helt korrekt. Kryssentropien ble estimert til å være 0.2083, som er lavt. Det at kryssentropien er lav kommer av at modellen ikke bare treffer riktig klasse, men samtidig gir høy sannsynlighet til de korrekte klassene. Her ser vi altså forskjellen mellom metrikkene accuracy som sier noe om modellen treffer feil eller riktig og der kryssentropien sier noe om hvor sikker og god sannsynlighetsfordelingen til modellen var.

3 - Klassifikasjon av MNIST med nevrale nettverk [Totalt 10p]

Klassen og funksjonene vi trenger skal nå være ferdig implementert, og det er på tide å bruke dem for å klassifisere de håndtegnede sifrene i MNIST.

Oppgave 3.1 [2p]: Tren modellen under og finn en god læringsrate og størrelse på mini-batchen. Bruk 10 epoker og send med evalueringssettet til treningen. Prøv å oppnå minst 90% accuracy på valideringssettet. Dere trenger kun å levere koden for treningen med de beste hyperparameterene dere finner. Bruk en modell med ett skjult lag med 16 noder og sigmoid aktiveringsfunksjon (dette er gitt i koden under).

Merk: Siden `softmax`, som er aktiveringsfunksjonen for siste lag, er brukt i tapsfunksjonen, skal siste aktiveringsfunksjon være "ingenting", som kan gjøres ved å bruke `IdentityActivation`. Det er implementert på denne måten for at man kan kunne bruke både vektorisert og ikke-vektorisert kode for aktiveringsfunksjonene.

```
# Laster inn MNIST-dataen og skalerer pikselverdiene til intervallet [0, 1].
# Grunnen til at dette gjøres her, kommer av at nevrale nettverk trener mer stabilt når inputverdiene ikke er for store, men bedre når det er mindre flyttall.
data = load_mnist_data(scale_x_data=True)
x_train = data["x_train"]
y_train = data["y_train"]
x_val = data["x_val"]
y_val = data["y_val"]

# Evaluerinssettet blir sendt til train() for loss og accuracy etter
```

```
hver epoke
eval_set = (x_val, y_val)

# Her lager vi et nevralt nettverk med 784 inputnoder, 16 noder i det
skjulte laget og
# 10 noder i outputlaget.
# Det skjulte laget bruker sigmoid som aktiveringsfunksjon og det
siste laget bruker
# IdentityActivation fordi softmax håndteres i tapsfunksjonen.
neural_network = NeuralNetwork(
    layer_sizes=[784, 16, 10],
    activation_functions=[Sigmoid(), IdentityActivation()],
)

# Her skal vi velge hyperparametere. Parameteret learning_rate styrer
hvor store steg
# modellen tar når vektene oppdateres, mens minibatch_size styrer hvor
mange datapunkter
# som brukes per parameteroppdatering
learning_rate = 0.1  # Virker naturlig med mange små steg i stedet for
få store
minibatch_size = 64  # Naturlig med 64 punkter her

# Siden forward(), predict(), loss og accuracy er implementert, så kan
train() kjøres slik
implemented = True

if implemented:
    neural_network.train(
        x_train=x_train,
        y_train=y_train,
        eta=learning_rate,
        loss_func=calculate_multiclass_cross_entropy,
        accuracy_func=calculate_accuracy,
        minibatch_size=minibatch_size,
        n_epochs=10,
        eval_set=eval_set,
    )
```

```
Epoch number [1 / 10]
Train-loss: 1.83848, Validation-loss: 1.83938. Train-accuracy:
0.85038, Validation-accuracy: 0.85340
Epoch number [2 / 10]
Train-loss: 1.78042, Validation-loss: 1.78285. Train-accuracy:
0.86836, Validation-accuracy: 0.87160
Epoch number [3 / 10]
Train-loss: 1.73947, Validation-loss: 1.74264. Train-accuracy:
0.88308, Validation-accuracy: 0.88470
Epoch number [4 / 10]
Train-loss: 1.70784, Validation-loss: 1.71118. Train-accuracy:
```

```
0.89296, Validation-accuracy: 0.89420
Epoch number [5 / 10]
Train-loss: 1.68588, Validation-loss: 1.68928. Train-accuracy:
0.89954, Validation-accuracy: 0.89970
Epoch number [6 / 10]
Train-loss: 1.67028, Validation-loss: 1.67383. Train-accuracy:
0.90358, Validation-accuracy: 0.90240
Epoch number [7 / 10]
Train-loss: 1.65904, Validation-loss: 1.66279. Train-accuracy:
0.90666, Validation-accuracy: 0.90450
Epoch number [8 / 10]
Train-loss: 1.65072, Validation-loss: 1.65469. Train-accuracy:
0.90852, Validation-accuracy: 0.90620
Epoch number [9 / 10]
Train-loss: 1.64427, Validation-loss: 1.64846. Train-accuracy:
0.91068, Validation-accuracy: 0.90890
Epoch number [10 / 10]
Train-loss: 1.63898, Validation-loss: 1.64337. Train-accuracy:
0.91212, Validation-accuracy: 0.91000
```

Forklaring oppgave 3.1

I denne oppgaven trente jeg et nevralt nettverk på MNIST med arkitekturen [784, 16, 10], altså ett skjult lag med 16 noder. Det skjulte laget brukte sigmoid som aktiveringsfunksjon, mens output-laget brukte IdentityActivation, siden softmax blir brukt i tapsfunksjonen. Jeg valgte læringsrate 0.1 og minibatch-størrelse på 64, og trente modellen i 10 epoker med valideringssettet som evalueringssett.

Resultatene viser at modellen lærte relativt jevnt gjennom hele treningen. Vi startet med epoke nummer 1 til å ha en validation-accuracy lik 0.85340, altså rundt 85% og denne økte gradvis for hver epoke, der valideringsnøyaktigheten var på nesten 90% på epoke 5. Etter den 10-ende og siste epoken endte valideringsnøyaktigheten på 91%, som oppnår minste-kravet.

Vi kan se at både treningstapet og valideringstapet sank jevnt fra starten til slutten av treningen, fra omtrent 1.84 til 1.64. Dette tyder på at modellen faktisk lærte bedre representasjoner av dataene etter hver epoke. Vi kan også observere at treningsaccuracy og valideringsaccuracy ligger svært nære hverandre gjennom hele treningssyklusen. Det er svært positivt, på grunn av at det viser at modellen generaliserer på en hensiktsmessig måte og ikke overtilpasser særlig.

For å konkludere, så virker det som at hyperparameterene jeg har valgt passer godt for modellen.

Oppgave 3.2 [4p]: Vi skal nå bruke plotte-funksjonene fra `plotting.py` til å visualisere noen av resultatene. Det kan hjelpe å lese doc-strengene (strengene i starten av funksjonene) til funksjonene fra filen `plotting.py` for å se hvilke argumenter de tar.

- a) Bruk `predict()` til å predikere på hele valideringssettet og lagre resultatene i en variabel.
- b) Bruk metoden `plot_random_mnist_images()` til å plotte bilder fra MNIST sammen med resultatene fra modellen. Send inn prediksjonene som argumentet

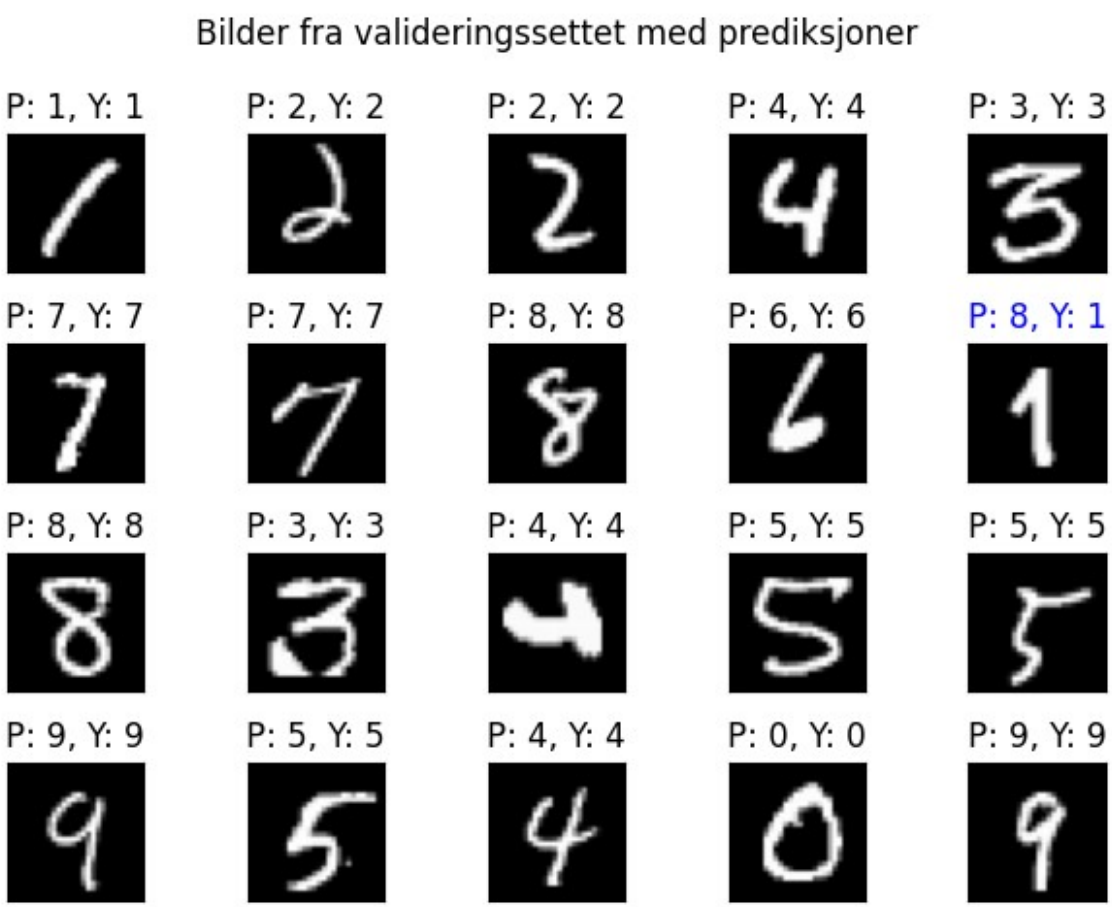
`predictions` og `y_val` som argumentet `labels`. Hvordan presterer modellen? Er de fleste sifrene predikert riktig?

- c) Bruk nå metoden `plot_mislabeled_mnist_images()` som kun vil plotte bilder som ble feilklassifisert av modellen. Hva synes du om ytelsen til modellen på disse bildene?
- d) Til slutt, bruk funksjonen `plot_worst_predicted_mnist_images()` som plotter de bildene der modellen predikerte mest feil. Her må inputet være logits-verdiene istedenfor prediksjonene, som returneres av `forward()`. Hva synes du om ytelsen til modellen på disse bildene? Likner disse bildene på bildene fra `b)`, eller skiller de seg ut?

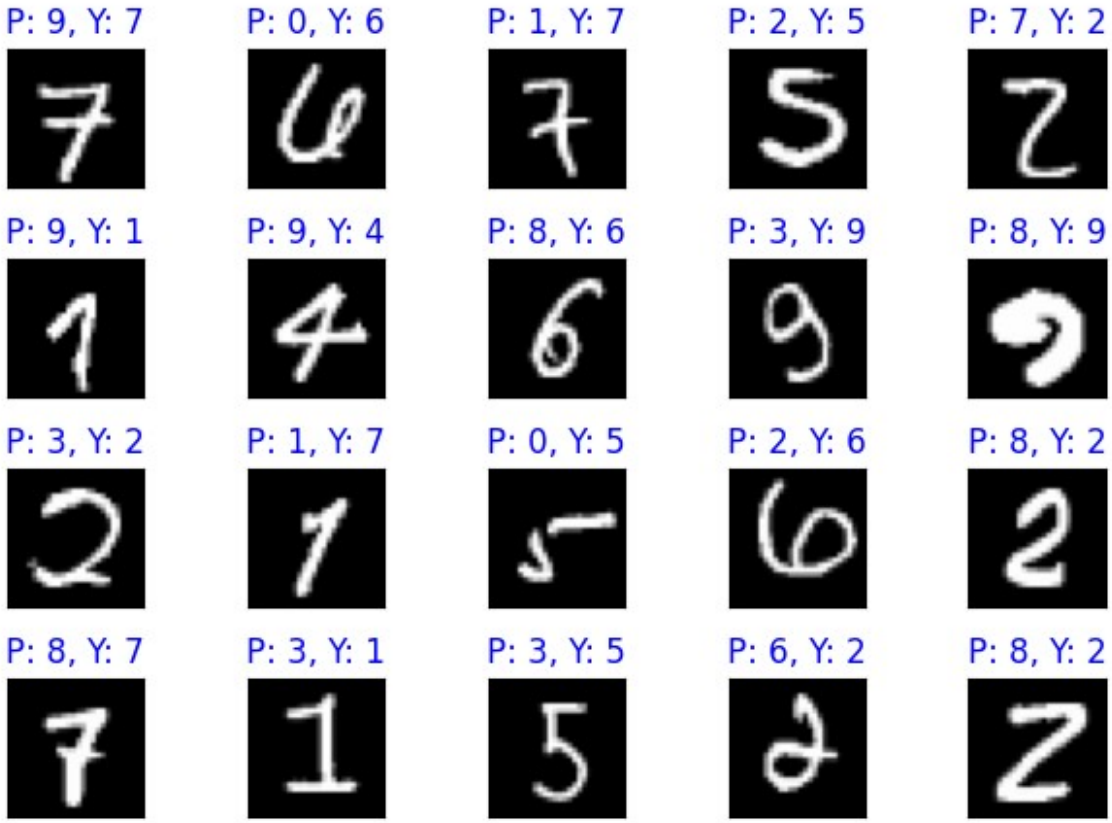
```
# Oppgave 3.2 a)
# Først bruker vi den ferdigtrente modellen til å predikere klasser
for hele
# valideringssettet. Funksjonen predict() returnerer heltallsklasser,
altså en
# predikert klasse per bilde, som [7, 2, 1, 0, ...]
val_predictions = neural_network.predict(x_val)

# Funksjonen forward() returnerer outputverdiene fra siste lag, før
softmax().
val_logits = neural_network.forward(x_val)

# Skriver ut shape for å kontrollere dimensjonene
print("Shape på prediksjoner:", val_predictions.shape)
print("Shape på logits:", val_logits.shape)

# Oppgave 3.2 b)
# Nå plottes de tilfeldige bildene fra valideringssettet, ved å sende
inn
# val_predictions for å vise prediksjonene og y_val slik at korrekt
klasse vises.
plot_random_mnist_images(
    images=x_val,
    predictions=val_predictions,
    labels=y_val,
    n_random=20,
    n_cols=5,
    title="Bilder fra valideringssettet med prediksjoner"
)

# Oppgave 3.2 c)
# Her plottes bildene som ble klassifisert feil, siden vi dermed kan
se
# hvilke type sifre modellen sliter med å klasifisere.
# Vi antar at dette vil være uklare, skjeve og sifre som ligner på
andre sifre.
plot_mislabeled_mnist_images(
    images=x_val,
    predictions=val_predictions,
    labels=y_val,
```

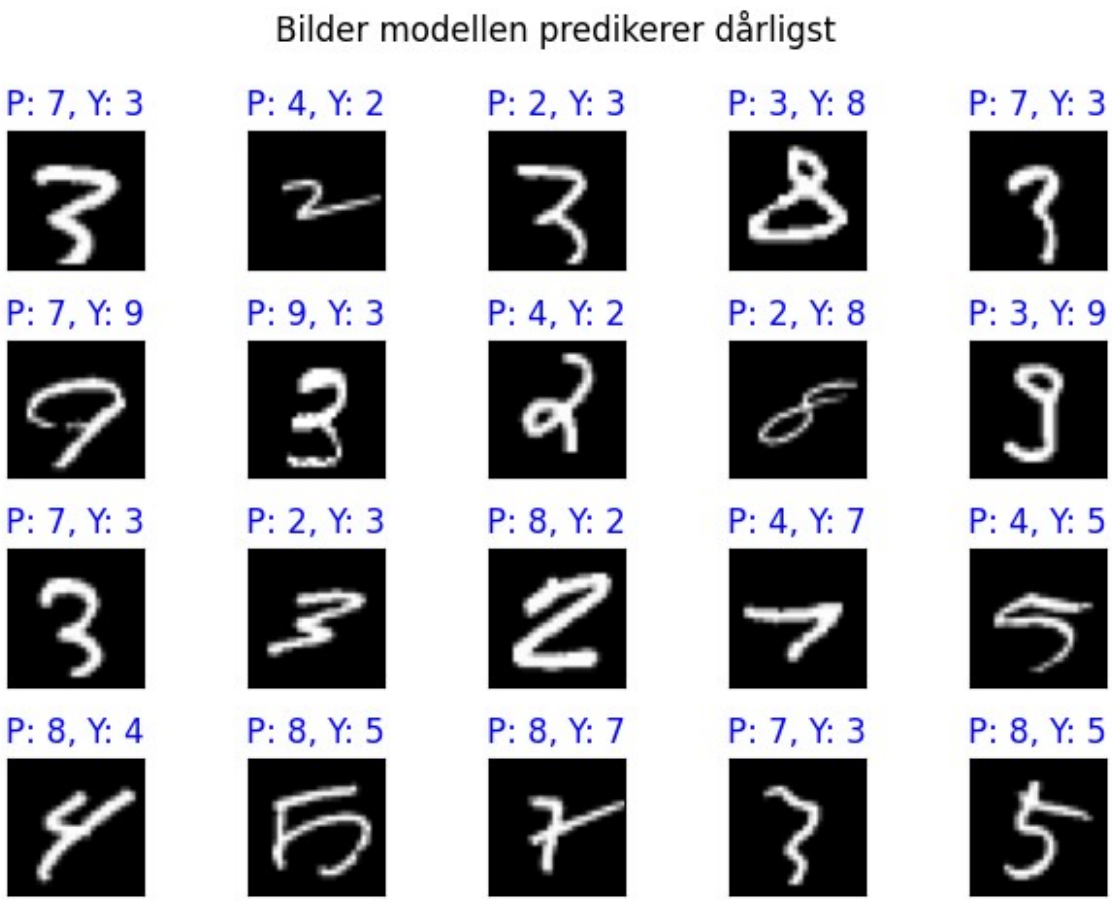
```
    n_random=20,
    n_cols=5,
    title="Feilklassifiserte bilder fra valideringssettet",
)

# Oppgave 3.2 d)
# Her plottes de verte prediksjonene modellen utførte. Denne
funksjonen trenger
# logits og ikke ferdige prediksjoner.
plot_worst_predicted_mnist_images(
    images=x_val,
    logits=val_logits,
    labels=y_val,
    n_images=20,
    n_cols=5,
    title="Bilder modellen predikerer dårligst"
)
```

```
Shape på prediksjoner: (10000,)
Shape på logits: (10000, 10)
```



Feilklassifiserte bilder fra valideringssettet





Forklaring oppgave 3.2

I denne oppgaven bruktes den ferdigtrente modellen til å predikere på hele valideringssettet med predict(), og lagret resultat i val_predictions. Jeg lagret logits fra forward() i val_logits, siden disse trengs for å finne de dårligste prediksjonene. Deretter brukte jeg plot_random_mnist_images() for å se tilfeldige bilder sammen med modellens prediksjoner og de sanne klassene.

Videre bruktes plot_mislabeled_mnist_images() for å undersøke bare de bildene modellen klasssifiserte feil. Disse bildene var vanskeligere å tolke, der sifrene er uklare, skjeve og ligner på andre sifre.

Til slutt bruktes plot_worst_predicted_mnist_images() med logits fra forward() for å finne de bildene modellen tok aller mest feil på. Disse bildene skiller seg mer ut enn de tilfeldige og er de vanskeligste eksemplene i datasettet.

La oss prøve å forbedre ytelsen til modellen vår. Vi har så langt bare brukt ett skjult lag. Siden MNIST er et enkelt datasett å klassifisere, kan man få god ytelse med dette, men la oss prøve med flere skjulte lag med flere noder.

Legg merke til at jo flere parametere modellen inneholder (fra flere skjulte lag og flere noder i disse lagene), jo lenger tar modellen å trene. Treningstiden vil variere en del fra datamaskin til

datamaskin, men prøv å bruke konfigurasjoner som gjør at modellen ikke bruker mer enn et par minutter til å trene ferdig i den endelige innleveringen.

Oppgave 3.3 [4p]: La oss prøve å forbedre modellen vår ved å endre på hyperparameterne for hvor mange skjulte lag vi bruker og hvor mange noder de lagene inneholder.

- a) Bruk to til tre skjulte lag og flere noder i lagene. Prøv å finn gode hyperparametere som gir høy nøyaktighet på valideringssettet (ca. 98% skal være oppnåelig). Bruk opp til 30 epoker i treningen.
- b) Prøv nå å trene uten noen skjulte lag. Hvor bra klarer dere å få denne modellen? Hvilken modell er dette ekvivalent med?
- c) Bruk ReLU som aktiveringsfunksjon og prøv å finne hyperparametere som gir best nøyaktighet på valideringssettet og rapporter resultatene. Dere kan også bruke en blanding av Sigmoid og ReLU.
- d) Test til slutt de beste modellene deres på testsettet og rapporter nøyaktigheten. Ikke gjør dette før dere her helt ferdig med de andre punktene og ikke har tenkt til å gjøre flere endringer i modellene deres. Hvordan er resultatet i forhold til trenings- og valideringssettene? Hvorfor tror dere det er slik?

Merk: Det er vanlig å ha flest noder i de tidlige skjulte lagene og færre for de senere, men dere kan prøve dere frem helt fritt.

Merk: For realistiske datasett er det essensielt med mange skjulte lag og mange parametere for at modellen skal trene bra, men siden MNIST er såpass enkelt, kan det være at dere får nesten like gode eller bedre resultater med bare ett skjult lag og få parametere.

```
# Laster inn datasettet med skalerte inputverdier.
# Dette gjør treningen mer stabil fordi pikselverdiene da ligger i
intervallet [0, 1].
data = load_mnist_data(scale_x_data=True)

x_train = data["x_train"]
y_train = data["y_train"]
x_val = data["x_val"]
y_val = data["y_val"]
x_test = data["x_test"]
y_test = data["y_test"]

# Valideringssettet brukes til å sammenligne modellene underveis i
treningen.
eval_set = (x_val, y_val)


def train_model(
    layer_sizes,
    activation_functions,
    learning_rate,
    minibatch_size,
    n_epochs,
    model_name,
```

```
):
    print(f"\n------------ Trener modell: {model_name} ------------")
    print(f"Layer sizes: {layer_sizes}")
    print(f"Learning rate: {learning_rate}")
    print(f"Mini-batch size: {minibatch_size}")
    print(f"Epochs: {n_epochs}\n")

    # Lager modellen med ønsket arkitektur.
    model = NeuralNetwork(
        layer_sizes=layer_sizes,
        activation_functions=activation_functions,
    )

    # Trener modellen og skriver ut loss/accuracy etter hver epoke.
    model.train(
        x_train=x_train,
        y_train=y_train,
        eta=learning_rate,
        loss_func=calculate_multiclass_cross_entropy,
        accuracy_func=calculate_accuracy,
        minibatch_size=minibatch_size,
        n_epochs=n_epochs,
        eval_set=eval_set,
    )

    return model

# Oppgave 3.3 a)
# Modell 1: To skjulte lag med ReLU
model_relu_better = train_model(
    layer_sizes=[784, 256, 128, 10],
    activation_functions=[ReLU(), ReLU(), IdentityActivation()],
    learning_rate=0.15,
    minibatch_size=32,
    n_epochs=30,
    model_name="To skjulte lag med ReLU, større modell",
)
```

```
------------ Trener modell: To skjulte lag med ReLU, større modell
------------
Layer sizes: [784, 256, 128, 10]
Learning rate: 0.15
Mini-batch size: 32
Epochs: 30

Epoch number [1 / 30]
Train-loss: 1.56579, Validation-loss: 1.57446. Train-accuracy:
0.95974, Validation-accuracy: 0.95090
Epoch number [2 / 30]
```

```
Train-loss: 1.53097, Validation-loss: 1.54149. Train-accuracy:
0.97262, Validation-accuracy: 0.96260
Epoch number [3 / 30]
Train-loss: 1.51936, Validation-loss: 1.53100. Train-accuracy:
0.97928, Validation-accuracy: 0.96860
Epoch number [4 / 30]
Train-loss: 1.51433, Validation-loss: 1.52691. Train-accuracy:
0.98330, Validation-accuracy: 0.97220
Epoch number [5 / 30]
Train-loss: 1.50435, Validation-loss: 1.51785. Train-accuracy:
0.98604, Validation-accuracy: 0.97470
Epoch number [6 / 30]
Train-loss: 1.49895, Validation-loss: 1.51359. Train-accuracy:
0.98816, Validation-accuracy: 0.97600
Epoch number [7 / 30]
Train-loss: 1.49708, Validation-loss: 1.51286. Train-accuracy:
0.98970, Validation-accuracy: 0.97750
Epoch number [8 / 30]
Train-loss: 1.49308, Validation-loss: 1.50984. Train-accuracy:
0.99080, Validation-accuracy: 0.97850
Epoch number [9 / 30]
Train-loss: 1.49006, Validation-loss: 1.50775. Train-accuracy:
0.99188, Validation-accuracy: 0.97860
Epoch number [10 / 30]
Train-loss: 1.48790, Validation-loss: 1.50645. Train-accuracy:
0.99252, Validation-accuracy: 0.97890
Epoch number [11 / 30]
Train-loss: 1.48531, Validation-loss: 1.50460. Train-accuracy:
0.99330, Validation-accuracy: 0.98000
Epoch number [12 / 30]
Train-loss: 1.48113, Validation-loss: 1.50143. Train-accuracy:
0.99390, Validation-accuracy: 0.98010
Epoch number [13 / 30]
Train-loss: 1.48100, Validation-loss: 1.50206. Train-accuracy:
0.99452, Validation-accuracy: 0.98060
Epoch number [14 / 30]
Train-loss: 1.47816, Validation-loss: 1.49999. Train-accuracy:
0.99510, Validation-accuracy: 0.98000
Epoch number [15 / 30]
Train-loss: 1.47676, Validation-loss: 1.49930. Train-accuracy:
0.99546, Validation-accuracy: 0.98040
Epoch number [16 / 30]
Train-loss: 1.47733, Validation-loss: 1.50068. Train-accuracy:
0.99588, Validation-accuracy: 0.98020
Epoch number [17 / 30]
Train-loss: 1.47540, Validation-loss: 1.49922. Train-accuracy:
0.99618, Validation-accuracy: 0.98060
Epoch number [18 / 30]
Train-loss: 1.47366, Validation-loss: 1.49821. Train-accuracy:
```

```
0.99652, Validation-accuracy: 0.98010
Epoch number [19 / 30]
Train-loss: 1.47344, Validation-loss: 1.49835. Train-accuracy:
0.99670, Validation-accuracy: 0.98030
Epoch number [20 / 30]
Train-loss: 1.47298, Validation-loss: 1.49849. Train-accuracy:
0.99694, Validation-accuracy: 0.98020
Epoch number [21 / 30]
Train-loss: 1.47326, Validation-loss: 1.49927. Train-accuracy:
0.99722, Validation-accuracy: 0.98050
Epoch number [22 / 30]
Train-loss: 1.47151, Validation-loss: 1.49798. Train-accuracy:
0.99748, Validation-accuracy: 0.98000
Epoch number [23 / 30]
Train-loss: 1.47343, Validation-loss: 1.50040. Train-accuracy:
0.99758, Validation-accuracy: 0.98040
Epoch number [24 / 30]
Train-loss: 1.47188, Validation-loss: 1.49933. Train-accuracy:
0.99772, Validation-accuracy: 0.98030
Epoch number [25 / 30]
Train-loss: 1.47220, Validation-loss: 1.50002. Train-accuracy:
0.99780, Validation-accuracy: 0.98020
Epoch number [26 / 30]
Train-loss: 1.47061, Validation-loss: 1.49897. Train-accuracy:
0.99782, Validation-accuracy: 0.98020
Epoch number [27 / 30]
Train-loss: 1.47152, Validation-loss: 1.50030. Train-accuracy:
0.99790, Validation-accuracy: 0.98010
Epoch number [28 / 30]
Train-loss: 1.47092, Validation-loss: 1.50015. Train-accuracy:
0.99798, Validation-accuracy: 0.98030
Epoch number [29 / 30]
Train-loss: 1.47118, Validation-loss: 1.50067. Train-accuracy:
0.99810, Validation-accuracy: 0.98030
Epoch number [30 / 30]
Train-loss: 1.47072, Validation-loss: 1.50047. Train-accuracy:
0.99818, Validation-accuracy: 0.98020
```

```python
# Modell 2: Tre skjulte lag med sigmoid
# Litt lavere læringsrate enn over kan være tryggere når modellen blir
dypere.
model_sigmoid_deeper = train_model(
    layer_sizes=[784, 128, 64, 32, 10],
    activation_functions=[Sigmoid(), Sigmoid(), Sigmoid(),
IdentityActivation()],
    learning_rate=0.05,
    minibatch_size=64,
    n_epochs=20,
    model_name="Tre skjulte lag med sigmoid",
)
```

```
------------ Trener modell: Tre skjulte lag med sigmoid ------------
Layer sizes: [784, 128, 64, 32, 10]
Learning rate: 0.05
Mini-batch size: 64
Epochs: 20

Epoch number [1 / 20]
Train-loss: 2.28959, Validation-loss: 2.28897. Train-accuracy:
0.22140, Validation-accuracy: 0.22660
Epoch number [2 / 20]
Train-loss: 2.25539, Validation-loss: 2.25457. Train-accuracy:
0.40350, Validation-accuracy: 0.41000
Epoch number [3 / 20]
Train-loss: 2.17683, Validation-loss: 2.17585. Train-accuracy:
0.46192, Validation-accuracy: 0.46590
Epoch number [4 / 20]
Train-loss: 2.08706, Validation-loss: 2.08652. Train-accuracy:
0.61182, Validation-accuracy: 0.61310
Epoch number [5 / 20]
Train-loss: 2.02508, Validation-loss: 2.02533. Train-accuracy:
0.69498, Validation-accuracy: 0.69450
Epoch number [6 / 20]
Train-loss: 1.98761, Validation-loss: 1.98845. Train-accuracy:
0.72976, Validation-accuracy: 0.73080
Epoch number [7 / 20]
Train-loss: 1.96250, Validation-loss: 1.96356. Train-accuracy:
0.75304, Validation-accuracy: 0.75690
Epoch number [8 / 20]
Train-loss: 1.93759, Validation-loss: 1.93859. Train-accuracy:
0.76742, Validation-accuracy: 0.76930
Epoch number [9 / 20]
Train-loss: 1.91162, Validation-loss: 1.91239. Train-accuracy:
0.77688, Validation-accuracy: 0.77780
Epoch number [10 / 20]
Train-loss: 1.88908, Validation-loss: 1.88968. Train-accuracy:
0.78746, Validation-accuracy: 0.78970
Epoch number [11 / 20]
Train-loss: 1.87116, Validation-loss: 1.87177. Train-accuracy:
0.79982, Validation-accuracy: 0.80180
Epoch number [12 / 20]
Train-loss: 1.85557, Validation-loss: 1.85628. Train-accuracy:
0.81276, Validation-accuracy: 0.81610
Epoch number [13 / 20]
Train-loss: 1.84071, Validation-loss: 1.84155. Train-accuracy:
0.82600, Validation-accuracy: 0.82740
Epoch number [14 / 20]
Train-loss: 1.82667, Validation-loss: 1.82767. Train-accuracy:
0.83800, Validation-accuracy: 0.84080
Epoch number [15 / 20]
```

```
Train-loss: 1.81380, Validation-loss: 1.81497. Train-accuracy:
0.84862, Validation-accuracy: 0.85140
Epoch number [16 / 20]
Train-loss: 1.80181, Validation-loss: 1.80311. Train-accuracy:
0.85808, Validation-accuracy: 0.85990
Epoch number [17 / 20]
Train-loss: 1.79009, Validation-loss: 1.79149. Train-accuracy:
0.86682, Validation-accuracy: 0.86770
Epoch number [18 / 20]
Train-loss: 1.77821, Validation-loss: 1.77967. Train-accuracy:
0.87466, Validation-accuracy: 0.87570
Epoch number [19 / 20]
Train-loss: 1.76617, Validation-loss: 1.76768. Train-accuracy:
0.88030, Validation-accuracy: 0.88290
Epoch number [20 / 20]
Train-loss: 1.75442, Validation-loss: 1.75599. Train-accuracy:
0.88396, Validation-accuracy: 0.88470
```

```python
# Oppgave 3.3 b)
# Modell uten skjulte lag.
model_no_hidden = train_model(
    layer_sizes=[784, 10],
    activation_functions=[IdentityActivation()],
    learning_rate=0.001,
    minibatch_size=64,
    n_epochs=20,
    model_name="Ingen skjulte lag",
)
```

```
------------ Trener modell: Ingen skjulte lag ------------
Layer sizes: [784, 10]
Learning rate: 0.001
Mini-batch size: 64
Epochs: 20

Epoch number [1 / 20]
Train-loss: 1.99298, Validation-loss: 1.99363. Train-accuracy:
0.63312, Validation-accuracy: 0.63800
Epoch number [2 / 20]
Train-loss: 1.93122, Validation-loss: 1.93241. Train-accuracy:
0.72764, Validation-accuracy: 0.72990
Epoch number [3 / 20]
Train-loss: 1.90427, Validation-loss: 1.90588. Train-accuracy:
0.76448, Validation-accuracy: 0.76460
Epoch number [4 / 20]
Train-loss: 1.88900, Validation-loss: 1.89095. Train-accuracy:
0.78332, Validation-accuracy: 0.78350
Epoch number [5 / 20]
Train-loss: 1.87912, Validation-loss: 1.88137. Train-accuracy:
```

```
0.79548, Validation-accuracy: 0.79730
Epoch number [6 / 20]
Train-loss: 1.87219, Validation-loss: 1.87469. Train-accuracy:
0.80420, Validation-accuracy: 0.80570
Epoch number [7 / 20]
Train-loss: 1.86702, Validation-loss: 1.86974. Train-accuracy:
0.80996, Validation-accuracy: 0.81150
Epoch number [8 / 20]
Train-loss: 1.86298, Validation-loss: 1.86589. Train-accuracy:
0.81572, Validation-accuracy: 0.81630
Epoch number [9 / 20]
Train-loss: 1.85971, Validation-loss: 1.86279. Train-accuracy:
0.81962, Validation-accuracy: 0.82050
Epoch number [10 / 20]
Train-loss: 1.85700, Validation-loss: 1.86023. Train-accuracy:
0.82280, Validation-accuracy: 0.82470
Epoch number [11 / 20]
Train-loss: 1.85469, Validation-loss: 1.85805. Train-accuracy:
0.82610, Validation-accuracy: 0.82680
Epoch number [12 / 20]
Train-loss: 1.85268, Validation-loss: 1.85616. Train-accuracy:
0.82836, Validation-accuracy: 0.82890
Epoch number [13 / 20]
Train-loss: 1.85092, Validation-loss: 1.85451. Train-accuracy:
0.83010, Validation-accuracy: 0.83060
Epoch number [14 / 20]
Train-loss: 1.84936, Validation-loss: 1.85304. Train-accuracy:
0.83192, Validation-accuracy: 0.83190
Epoch number [15 / 20]
Train-loss: 1.84795, Validation-loss: 1.85172. Train-accuracy:
0.83354, Validation-accuracy: 0.83330
Epoch number [16 / 20]
Train-loss: 1.84667, Validation-loss: 1.85053. Train-accuracy:
0.83442, Validation-accuracy: 0.83400
Epoch number [17 / 20]
Train-loss: 1.84550, Validation-loss: 1.84944. Train-accuracy:
0.83524, Validation-accuracy: 0.83440
Epoch number [18 / 20]
Train-loss: 1.84443, Validation-loss: 1.84844. Train-accuracy:
0.83636, Validation-accuracy: 0.83510
Epoch number [19 / 20]
Train-loss: 1.84344, Validation-loss: 1.84751. Train-accuracy:
0.83730, Validation-accuracy: 0.83640
Epoch number [20 / 20]
Train-loss: 1.84252, Validation-loss: 1.84666. Train-accuracy:
0.83834, Validation-accuracy: 0.83740
```

```
# Oppgave 3.3 c)
# Modell 3: To skjulte lag med ReLU
model_relu = train_model(
```

```
    layer_sizes=[784, 128, 64, 10],
    activation_functions=[ReLU(), ReLU(), IdentityActivation()],
    learning_rate=0.001,
    minibatch_size=64,
    n_epochs=20,
    model_name="To skjulte lag med ReLU",
)
```

```
------------ Trener modell: To skjulte lag med ReLU ------------
Layer sizes: [784, 128, 64, 10]
Learning rate: 0.001
Mini-batch size: 64
Epochs: 20

Epoch number [1 / 20]
Train-loss: 2.15997, Validation-loss: 2.16083. Train-accuracy:
0.48648, Validation-accuracy: 0.48810
Epoch number [2 / 20]
Train-loss: 2.07411, Validation-loss: 2.07520. Train-accuracy:
0.61900, Validation-accuracy: 0.62260
Epoch number [3 / 20]
Train-loss: 2.01860, Validation-loss: 2.01986. Train-accuracy:
0.69272, Validation-accuracy: 0.69520
Epoch number [4 / 20]
Train-loss: 1.97941, Validation-loss: 1.98081. Train-accuracy:
0.73880, Validation-accuracy: 0.74420
Epoch number [5 / 20]
Train-loss: 1.94875, Validation-loss: 1.95039. Train-accuracy:
0.76750, Validation-accuracy: 0.77120
Epoch number [6 / 20]
Train-loss: 1.92377, Validation-loss: 1.92558. Train-accuracy:
0.78884, Validation-accuracy: 0.79230
Epoch number [7 / 20]
Train-loss: 1.90301, Validation-loss: 1.90497. Train-accuracy:
0.80580, Validation-accuracy: 0.80920
Epoch number [8 / 20]
Train-loss: 1.88552, Validation-loss: 1.88759. Train-accuracy:
0.81924, Validation-accuracy: 0.82160
Epoch number [9 / 20]
Train-loss: 1.87037, Validation-loss: 1.87253. Train-accuracy:
0.82982, Validation-accuracy: 0.83170
Epoch number [10 / 20]
Train-loss: 1.85717, Validation-loss: 1.85939. Train-accuracy:
0.83968, Validation-accuracy: 0.84020
Epoch number [11 / 20]
Train-loss: 1.84538, Validation-loss: 1.84765. Train-accuracy:
0.84698, Validation-accuracy: 0.84800
Epoch number [12 / 20]
Train-loss: 1.83478, Validation-loss: 1.83713. Train-accuracy:
```

```
0.85294, Validation-accuracy: 0.85410
Epoch number [13 / 20]
Train-loss: 1.82511, Validation-loss: 1.82752. Train-accuracy:
0.85772, Validation-accuracy: 0.85890
Epoch number [14 / 20]
Train-loss: 1.81633, Validation-loss: 1.81880. Train-accuracy:
0.86234, Validation-accuracy: 0.86380
Epoch number [15 / 20]
Train-loss: 1.80812, Validation-loss: 1.81065. Train-accuracy:
0.86606, Validation-accuracy: 0.86840
Epoch number [16 / 20]
Train-loss: 1.80057, Validation-loss: 1.80315. Train-accuracy:
0.87010, Validation-accuracy: 0.87150
Epoch number [17 / 20]
Train-loss: 1.79353, Validation-loss: 1.79617. Train-accuracy:
0.87298, Validation-accuracy: 0.87430
Epoch number [18 / 20]
Train-loss: 1.78697, Validation-loss: 1.78968. Train-accuracy:
0.87618, Validation-accuracy: 0.87710
Epoch number [19 / 20]
Train-loss: 1.78079, Validation-loss: 1.78357. Train-accuracy:
0.87946, Validation-accuracy: 0.87930
Epoch number [20 / 20]
Train-loss: 1.77497, Validation-loss: 1.77781. Train-accuracy:
0.88172, Validation-accuracy: 0.88280
```

```python
# Modell 4: Tre skjulte lag med ReLU
model_relu_deeper = train_model(
    layer_sizes=[784, 256, 128, 64, 10],
    activation_functions=[ReLU(), ReLU(), ReLU(),
IdentityActivation()],
    learning_rate=0.0005,
    minibatch_size=64,
    n_epochs=20,
    model_name="Tre skjulte lag med ReLU",
)
```

```
------------ Trener modell: Tre skjulte lag med ReLU ------------
Layer sizes: [784, 256, 128, 64, 10]
Learning rate: 0.0005
Mini-batch size: 64
Epochs: 20

Epoch number [1 / 20]
Train-loss: 2.27221, Validation-loss: 2.27230. Train-accuracy:
0.21006, Validation-accuracy: 0.20710
Epoch number [2 / 20]
Train-loss: 2.24262, Validation-loss: 2.24284. Train-accuracy:
0.35894, Validation-accuracy: 0.36030
```

```
Epoch number [3 / 20]
Train-loss: 2.21236, Validation-loss: 2.21283. Train-accuracy:
0.45826, Validation-accuracy: 0.45960
Epoch number [4 / 20]
Train-loss: 2.18114, Validation-loss: 2.18184. Train-accuracy:
0.53932, Validation-accuracy: 0.53250
Epoch number [5 / 20]
Train-loss: 2.15022, Validation-loss: 2.15113. Train-accuracy:
0.60048, Validation-accuracy: 0.59740
Epoch number [6 / 20]
Train-loss: 2.12050, Validation-loss: 2.12163. Train-accuracy:
0.64780, Validation-accuracy: 0.64850
Epoch number [7 / 20]
Train-loss: 2.09240, Validation-loss: 2.09367. Train-accuracy:
0.68348, Validation-accuracy: 0.68110
Epoch number [8 / 20]
Train-loss: 2.06614, Validation-loss: 2.06750. Train-accuracy:
0.71076, Validation-accuracy: 0.70940
Epoch number [9 / 20]
Train-loss: 2.04182, Validation-loss: 2.04326. Train-accuracy:
0.73258, Validation-accuracy: 0.73170
Epoch number [10 / 20]
Train-loss: 2.01947, Validation-loss: 2.02101. Train-accuracy:
0.75032, Validation-accuracy: 0.75130
Epoch number [11 / 20]
Train-loss: 1.99915, Validation-loss: 2.00079. Train-accuracy:
0.76458, Validation-accuracy: 0.76580
Epoch number [12 / 20]
Train-loss: 1.98076, Validation-loss: 1.98248. Train-accuracy:
0.77624, Validation-accuracy: 0.77900
Epoch number [13 / 20]
Train-loss: 1.96409, Validation-loss: 1.96590. Train-accuracy:
0.78580, Validation-accuracy: 0.78960
Epoch number [14 / 20]
Train-loss: 1.94891, Validation-loss: 1.95083. Train-accuracy:
0.79416, Validation-accuracy: 0.79870
Epoch number [15 / 20]
Train-loss: 1.93506, Validation-loss: 1.93709. Train-accuracy:
0.80218, Validation-accuracy: 0.80650
Epoch number [16 / 20]
Train-loss: 1.92231, Validation-loss: 1.92444. Train-accuracy:
0.80936, Validation-accuracy: 0.81370
Epoch number [17 / 20]
Train-loss: 1.91054, Validation-loss: 1.91275. Train-accuracy:
0.81618, Validation-accuracy: 0.82060
Epoch number [18 / 20]
Train-loss: 1.89962, Validation-loss: 1.90188. Train-accuracy:
0.82162, Validation-accuracy: 0.82580
Epoch number [19 / 20]
```

```
Train-loss: 1.88948, Validation-loss: 1.89177. Train-accuracy:
0.82720, Validation-accuracy: 0.83220
Epoch number [20 / 20]
Train-loss: 1.88003, Validation-loss: 1.88233. Train-accuracy:
0.83296, Validation-accuracy: 0.83770
```

```
# Modell 5: Blandet aktivering - ReLU tidlig, sigmoid senere
model_mixed = train_model(
    layer_sizes=[784, 128, 64, 10],
    activation_functions=[ReLU(), Sigmoid(), IdentityActivation()],
    learning_rate=0.001,
    minibatch_size=64,
    n_epochs=20,
    model_name="Blandet ReLU og Sigmoid",
)
```

```
------------ Trener modell: Blandet ReLU og Sigmoid ------------
Layer sizes: [784, 128, 64, 10]
Learning rate: 0.001
Mini-batch size: 64
Epochs: 20

Epoch number [1 / 20]
Train-loss: 2.26837, Validation-loss: 2.26855. Train-accuracy:
0.33316, Validation-accuracy: 0.33380
Epoch number [2 / 20]
Train-loss: 2.24068, Validation-loss: 2.24086. Train-accuracy:
0.44934, Validation-accuracy: 0.44880
Epoch number [3 / 20]
Train-loss: 2.21441, Validation-loss: 2.21458. Train-accuracy:
0.50916, Validation-accuracy: 0.51160
Epoch number [4 / 20]
Train-loss: 2.18902, Validation-loss: 2.18918. Train-accuracy:
0.55092, Validation-accuracy: 0.55270
Epoch number [5 / 20]
Train-loss: 2.16477, Validation-loss: 2.16492. Train-accuracy:
0.58154, Validation-accuracy: 0.58560
Epoch number [6 / 20]
Train-loss: 2.14203, Validation-loss: 2.14218. Train-accuracy:
0.60764, Validation-accuracy: 0.61110
Epoch number [7 / 20]
Train-loss: 2.12097, Validation-loss: 2.12113. Train-accuracy:
0.62936, Validation-accuracy: 0.63300
Epoch number [8 / 20]
Train-loss: 2.10164, Validation-loss: 2.10184. Train-accuracy:
0.65108, Validation-accuracy: 0.65300
Epoch number [9 / 20]
Train-loss: 2.08395, Validation-loss: 2.08422. Train-accuracy:
0.66962, Validation-accuracy: 0.66990
```

```
Epoch number [10 / 20]
Train-loss: 2.06778, Validation-loss: 2.06812. Train-accuracy:
0.68714, Validation-accuracy: 0.68650
Epoch number [11 / 20]
Train-loss: 2.05298, Validation-loss: 2.05339. Train-accuracy:
0.70336, Validation-accuracy: 0.70310
Epoch number [12 / 20]
Train-loss: 2.03937, Validation-loss: 2.03985. Train-accuracy:
0.71820, Validation-accuracy: 0.71840
Epoch number [13 / 20]
Train-loss: 2.02679, Validation-loss: 2.02734. Train-accuracy:
0.73012, Validation-accuracy: 0.73070
Epoch number [14 / 20]
Train-loss: 2.01512, Validation-loss: 2.01574. Train-accuracy:
0.74254, Validation-accuracy: 0.74160
Epoch number [15 / 20]
Train-loss: 2.00423, Validation-loss: 2.00492. Train-accuracy:
0.75348, Validation-accuracy: 0.75230
Epoch number [16 / 20]
Train-loss: 1.99405, Validation-loss: 1.99481. Train-accuracy:
0.76170, Validation-accuracy: 0.76140
Epoch number [17 / 20]
Train-loss: 1.98450, Validation-loss: 1.98533. Train-accuracy:
0.77130, Validation-accuracy: 0.77030
Epoch number [18 / 20]
Train-loss: 1.97553, Validation-loss: 1.97642. Train-accuracy:
0.77922, Validation-accuracy: 0.77910
Epoch number [19 / 20]
Train-loss: 1.96708, Validation-loss: 1.96804. Train-accuracy:
0.78586, Validation-accuracy: 0.78650
Epoch number [20 / 20]
Train-loss: 1.95913, Validation-loss: 1.96015. Train-accuracy:
0.79206, Validation-accuracy: 0.79200
```

```python
# Oppgave 3.3 d)
# Den beste modellen var den med to skjulte lag med ReLU og flere
noder
best_model = model_relu_better

# Predikerer på testsettet
test_predictions = best_model.predict(x_test)

# Beregner testnøyaktighet
test_accuracy = calculate_accuracy(y_test, test_predictions)

print("Test accuracy:", test_accuracy)


# Enkel oversikt over de beste valideringsaccuracy-ene for hver modell
results = {
```

```
    "To skjulte lag med ReLU, større modell": 0.98020,
    "Tre skjulte lag med sigmoid": 0.88470,
    "Ingen skjulte lag": 0.83740,
    "To skjulte lag med ReLU": 0.88280,
    "Tre skjulte lag med ReLU": 0.83770,
    "Blandet ReLU og Sigmoid": 0.79200,
}

results
```

```
Test accuracy: 0.9803
```

```
{'To skjulte lag med ReLU, større modell': 0.9802,
 'Tre skjulte lag med sigmoid': 0.8847,
 'Ingen skjulte lag': 0.8374,
 'To skjulte lag med ReLU': 0.8828,
 'Tre skjulte lag med ReLU': 0.8377,
 'Blandet ReLU og Sigmoid': 0.792}
```

Forklaring oppgave 3.3

I denne oppgaven ble det undersøkt hvordan antall skjulte lag, antall noder og valg av aktiveringsfunksjon påvirket ytelsen til et nevralt nettverk på MNIST. Målet var altså å finne en modell som klassifiserte sifrene så presist som mulig på valideringssettet, og til slutt teste modellen på testsettet.

I deloppgave 3.3a, ble dypere modeller med to og tre skjulte lag testet. Den beste modellen blant konfigurasjonene jeg prøvde var et nettverk med to skjulte lag og ReLU, med lagstørrelser [784, 256, 128, 10], læringsrate 0.15, mini-batch størrelse 32 og 30 epoker. Denne modellen oppnådde 0.98020 accuracy på valideringssettet, altså 98%. Konklusjonen er at en større modell med mer optimale hyperparametere kan gi svært høy ytelse på MNIST datasettet.

I deloppgave 3.3b, så ble det trent en modell uten skjulte lag, med arkitekturen [784, 10]. Den oppnådde 0.83740 accuracy på valideringssettet, altså 84%. Denne modellen er lik en multiklasse logistisk regresjon, på grunn av at den kun gjør én lineær transformasjon fra input dataene til output dataene.

I deloppgave 3.3c, så ble det testet modeller med ReLU, samt en blanding av ReLU og Sigmoid. Av de modell konfigurasjonene jeg testet, så endte den større ReLU modellen med best valideringsnøyaktighet. 0.88470 for tre skjulte lag med sigmoid og 0.79200 for den blandede modell konfigurasjonen. Det er viktig å bemerke seg at dette ikke nødvendigvis betyr at de andre arkitekturene er mindre optimale, det betyr kun at de ikke presterte like godt for mine tester, med mine foreløpige hyperparametere.

I deloppgave 3.3d, så ble den beste testmodellen testet på testsettet. Den oppnådde 0.9803 accuracy, som er svært nærme resultat fra valideringen. Det er dermed logisk å kunne konkludere med at modellen generaliserer godt til nye typer data og at valideringssettet ga et realistisk bilde av modellens faktiske ytelse.

[Bonus] Dyplæringsrammeverk: Det finnes flere *dyplæringsrammeverk* som gjør det enklere, mer effektivt og mer standardisert å trene nevrale nettverk. De mest populære er *PyTorch*

utviklet av Meta og *TensorFlow* utviklet av Google, der PyTorch har blitt den mest populære de siste årene. Det har også kommet nye rammeverk som blitt populære, som *Jax* og *TinyGrad*. Rammeverkene er store biblioteker man kan bruke i Python, som implementerer det meste av funksjonalitet relatert til dyplæring. Trening av store maskinlæringsmetoder kan være svært krevende med hensyn på beregning og minne, så derfor har alle dyplæringsrammeverk sterkt fokus på effektiv implementasjon og algoritmer. Det meste av koden som utfører beregninger er skrevet i C++, som er ekstremt mye raskere enn Python. Istedenfor NumPy-arrays blir det brukt *tensorer* som den fundamentale datastrukturen, som er spesialisert for dyplæring og kan dermed øke effektiviteten med omtrent 10–100 ganger sammenliknet med arrays. Algoritmene som blir brukt for matrisemultiplikasjon og beregninger er parallellisert og optimalisert.

[Bonus] Automatisk derivering: Et av de viktigste "triksene" for effektiv trening av nevrale nettverk er *automatisk derivering*. Den klassiske måten å gjøre gradientnedstigning på er å regne ut den deriverte (til tapsfunksjonen med hensyn på parameterne) for hånd og bruke dette til å finne ut en formel for oppdateringsregelen. Automatisk derivering gjør at man slipper å regne ut uttrykk av de deriverte direkte, i tillegg til at derivasjonen blir utført mye mer effektivt med hensyn på beregninger og minne. Det baserer seg på å loggføre alle beregninger som blir gjort med tensorer i en "*computational graph*". For å finne de deriverte kan man traversere grafen og følge derivasjonsregler, som skjer automatisk. For å lære mer om automatisk derivering er denne praktiske gjennomgangen av Andrej Karpathy sterkt anbefalt. Matematikken bak metoden blir forklart i denne videoen (her brukes litt mer lineær algebra enn vi gjør i dette kurset).

[Bonus] Litt dyplæringshistorie: Selv om kunstige nevrale nettverk har vært studert siden 50-tallet og de første applikasjonene med tilbakepropagering ble publisert i 1986, ble ikke dyplæring popularisert før i 2012. Nevrale nettverk klarte å klassifisere på datasett som MNIST lenge før dette, men resultatene var ikke imponerende nok til å popularisere modellene i noen særlig grad. For at nevrale nettverk skulle predikere effektivt på store og krevende datasett måtte flere brikker på plass. Flere av dem var relatert til triks for trening og implementasjon av modellene. Videre var hardware en viktig faktor: man fant ut at grafikkort kunne utføre beregningene i nevrale nettverk mye mer effektivt enn vanlige prosessorer. Dyplæring krever ofte store datasett for å bli effektive, så en annen viktig faktor var at større og bedre datasett ble konstruert og tilgjengeliggjort. Maskinssynkonkurransen *ImageNet* ble lansert i 2010, som inneholdt over en million bilder med menneskelig annoterte sanne verdier fra ti tusen klasser. I 2012 vant det nevrale nettverket som senere ble kalt for *AlexNet* konkurransen, som startet en stor internasjonal popularisering av dyplæring i domener som maskinssyn, lydprosessering og tekstprosessering.

Oppsummering

La oss oppsummere hva vi har sett på i denne obligen. Vi har brukt et enkelt forovermatet nevralt nettverk til å klassifisere bilder i MNIST datasettet. Modellen vår er implementert fra bunnen av med NumPy-arrays. Vi har implementert flere aktiveringsfunksjoner og deres deriverte, i tillegg til foroverfasen for de nevrale nettverkene og metrikker for trening. Vi så også på hva tilbakepropagering gjør og hva mini-batch SGD er, men det ble ikke implementert som del av denne obligen. Til slutt trente vi på MNIST datasettet med flere konfigurasjoner og tolket resultatet.

Denne obligen er ment for å gitt innsyn i hvordan nevrale nettverk fungerer på et detaljert nivå. Dersom dere lærer mer om dyplæring vil dere lære mer avanserte teknikker og bruk av dyplæringsrammeverk som abstraherer bort mye av det som skjer i bakgrunnen. De grunnleggende prinsippene, som forover- og bakoverfasen, tilbakepropagering og mini-batch SGD er allikevel tilsvarende, så det vil hjelpe å ha brukt tid på en lav-nivå implementasjon av nevrale nettverk for å forstå hva som skjer i bakgrunnen i mer avanserte kontekster.

For studenter med litt kunnskap om lineær algebra og derviasjon kan det være en veldig god oppgave og implementere tilbakepropagering og mini-batch SGD selv. Andre kapittel i denne boken går igjennom mesteparten av utregningene, og dere kan bruke implementasjonen i `utils2b.py` som hjelp. I `IN3310` vil dere lære mer om nevral nettverk for maskinsyn og `IN5550` dekker dyplæring for språkprosessering. Resten av videoserien til 3Blue1Brown går igjennom moderne dyplæringsarkitekturer for språkprosessering og er en tilgjengelig måte å lære mer avansert dyplæring på. Dersom dere vil bruke moderne dyplæringsrammeverk til å trene nevrale nettverk effektiv kan dere starte med dokumentasjonen til PyTorch.